



PDF Download
3639471.pdf
06 February 2026
Total Citations: 6
Total Downloads:
5569

Latest updates: <https://dl.acm.org/doi/10.1145/3639471>

RESEARCH-ARTICLE

Bridging Software-Hardware for CXL Memory Disaggregation in Billion-Scale Nearest Neighbor Search

Published: 19 February 2024

Online AM: 06 January 2024

Accepted: 15 December 2023

Received: 11 October 2023

[Citation in BibTeX format](#)

JUNHYEOK JANG, Korea Advanced Institute of Science and Technology, Daejeon, South Korea

HANJIN CHOI, Korea Advanced Institute of Science and Technology, Daejeon, South Korea

HANYEOREUM BAE, Korea Advanced Institute of Science and Technology, Daejeon, South Korea

SEUNGJUN LEE, Korea Advanced Institute of Science and Technology, Daejeon, South Korea

MIRYEONG KWON, Korea Advanced Institute of Science and Technology, Daejeon, South Korea

MYOUNGSOO JUNG, Korea Advanced Institute of Science and Technology, Daejeon, South Korea

Open Access Support provided by:

Korea Advanced Institute of Science and Technology

Bridging Software-Hardware for CXL Memory Disaggregation in Billion-Scale Nearest Neighbor Search

JUNHYEOK JANG, KAIST, Daejeon, Korea

HANJIN CHOI, KAIST and Panmnnesia, Inc., Daejeon, Korea

HANYEOREUM BAE and SEUNGJUN LEE, KAIST, Daejeon, Korea

MIRYEONG KWON and MYOUNGSOO JUNG, KAIST and Panmnnesia, Inc., Daejeon, Korea

We propose *CXL-ANNS*, a software-hardware collaborative approach to enable scalable approximate nearest neighbor search (ANNS) services. To this end, we first disaggregate DRAM from the host via compute express link (CXL) and place all essential datasets into its memory pool. While this CXL memory pool allows ANNS to handle billion-point graphs without an accuracy loss, we observe that the search performance significantly degrades because of CXL's far-memory-like characteristics. To address this, *CXL-ANNS* considers the node-level relationship and caches the neighbors in local memory, which are expected to visit most frequently. For the uncached nodes, *CXL-ANNS* prefetches a set of nodes most likely to visit soon by understanding the graph traversing behaviors of ANNS. *CXL-ANNS* is also aware of the architectural structures of the CXL interconnect network and lets different hardware components collaborate with each other for the search. Furthermore, it relaxes the execution dependency of neighbor search tasks and allows ANNS to utilize all hardware in the CXL network in parallel.

Our evaluation shows that *CXL-ANNS* exhibits 93.3% lower query latency than state-of-the-art ANNS platforms that we tested. *CXL-ANNS* also outperforms an oracle ANNS system that has unlimited local DRAM capacity by 68.0%, in terms of latency.

CCS Concepts: • **Hardware** → **Hardware accelerators; Hardware-software codesign**; • **Information systems** → **Nearest-neighbor search**;

Additional Key Words and Phrases: Approximate Nearest Neighbor Search (ANNS), software/hardware co-design, Compute Express Link (CXL)

ACM Reference Format:

Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2024. Bridging Software-Hardware for CXL Memory Disaggregation in Billion-Scale Nearest Neighbor Search. *ACM Trans. Storage* 20, 2, Article 10 (February 2024), 30 pages. <https://doi.org/10.1145/3639471>

1 INTRODUCTION

Dense retrieval (also known as nearest neighbor search) has taken on an important role and provides fundamental support for various search engines, data mining, databases, and machine

The authors thank anonymous reviewers for their constructive feedback as well as Panmnnesia for their technical support. This work is supported by Panmnnesia and protected by one or more patents.

Authors' addresses: J. Jang, H. Bae, and S. Lee, KAIST, 291, Daehak-ro, Yuseong-gu, Daejeon, Republic of Korea (34141); e-mails: jhjang@camelab.org, hyr.bae@camelab.org, sjlee@camelab.org; H. Choi, M. Kwon, and M. Jung (Corresponding author), KAIST and Panmnnesia, Inc.: 42, Hanbat-daero 371beon-gil, Yuseong-gu, Daejeon, Republic of Korea (34136) and 38, Yeongdong-daero 85-gil, Gangnam-gu, Seoul (06180); e-mails: hjchoi@camelab.org, mkwon@camelab.org, mj@camelab.org.



This work is licensed under a Creative Commons Attribution-NonCommercial International 4.0 License.

© 2024 Copyright held by the owner/author(s).

ACM 1553-3077/2024/02-ART10

<https://doi.org/10.1145/3639471>

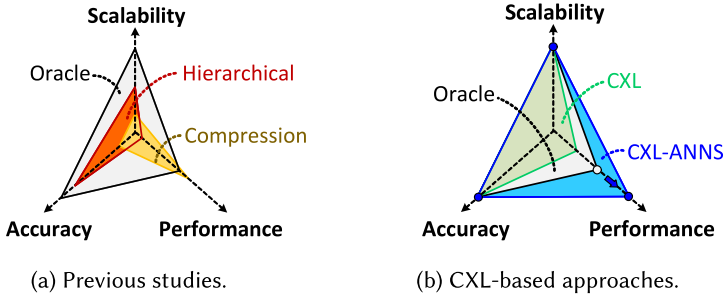


Fig. 1. Various billion-scale ANNS characterizations.

learning applications such as recommendation systems [10, 26, 42, 45, 65, 70–72]. In contrast to the classic pattern/string-based search, dense retrieval compares the similarity across different objects using their distance and retrieves a given number of objects, similar to the query object, referred to as *k*-**nearest neighbor** (*k*NN) [16, 18, 48]. To this end, dense retrieval embeds input information into a few thousand-dimensional spaces of each object called a feature *vector*. Since these vectors can encode a wide spectrum of data formats (images, documents, sounds, etc.), dense retrieval understands an input query’s semantics, resulting in more context-aware and accurate results than traditional search [26, 52, 63].

Even though *k*NN is one of the most frequently used search paradigms in various applications, it is a costly operation taking linear time to scan data [28, 67]. This computation complexity unfortunately makes dense retrieval with a billion-point dataset infeasible. To make the *k*NN search more practical, *approximate nearest neighbor search* (ANNS) restricts a query vector to search only a subset of neighbors with a high chance of being the nearest ones [4, 28, 46]. ANNS exhibits good vector searching speed and accuracy, but it significantly increases memory requirement and pressure. For example, many production-level recommendation systems already adopt billion-point datasets, which require tens of TB of working memory space for ANNS; Microsoft search engines (used in Bing/Outlook) require 100 B+ vectors, each being explained by 100 dimensions, which consume more than 40 TB memory space [57]. Similarly, several of Alibaba’s e-commerce platforms need TB-scale memory spaces to accommodate their 2 B+ vectors (128 dimensions) [15].

To address these memory pressure issues, modern ANNS techniques leverage lossy compression methods or employ persistent storage, such as **solid state disks (SSDs)** and **persistent memory (PMEM)**, for their memory expansion. For example, [6, 17, 21, 32] split large datasets and group them into multiple clusters in an offline time. This compression approach only has product quantized vectors for each cluster’s centroid and searches *k*NN based on the quantized information, making billion-scale ANNS feasible. On the other hand, the hierarchical approach [9, 19, 30, 56, 59] accommodates the datasets to SSD/PMEM, but reduces target search spaces by referring to a summary in its local memory (DRAM). As shown in Figure 1(a), these compression and hierarchical approaches can achieve the best *k*NN search performance and scalability similar to or slightly worse than what an *oracle*¹ system offers. However, these approaches suffer from a lack of accuracy and/or performance, which unfortunately hinders their practicality in achieving billion-scale ANNS services.

In this work, we propose *CXL-ANNS*, a software-hardware collaborative approach that enables scalable ANNS. As shown in Figure 1(b), the main goal of *CXL-ANNS* is to offer the latency of

¹In this article, the term “Oracle” refers to a system that utilizes ample DRAM resources with an unrestricted memory capacity.

billion-point kNN search even shorter than the oracle system mentioned above while achieving high throughput without a loss of accuracy. To this end, we disaggregate DRAM from the host resources via **compute express link (CXL)** and place all essential datasets into its memory pool; CXL is an open-industry interconnect technology that allows the underlying working memory to be highly scalable and composable with a low cost. Since a CXL network can expand its memory capacity by having more *endpoint devices*² (EPs) in a scalable manner, a host's **root-complex (RC)** can map the network's large memory pool (up to 4 PB) into its system memory space and use it just like a locally-attached conventional DRAM.

While this CXL memory pool can make ANNS feasible to handle billion-point graphs without a loss of accuracy, we observe that the search performance degrades compared to the oracle by as high as 3.9× (Section 3.1). This is due to CXL's far-memory-like characteristics; every memory request needs a CXL protocol conversion (from CPU instructions to one or more CXL flits), which takes a time similar to or longer than a DRAM access itself. To address this, we consider the relationship of different nodes in a given graph and cache the neighbors in the local memory, which are expected to visit frequently. For the uncached nodes, CXL-ANNS prefetches a set of nodes most likely to be touched soon by understanding the unique behaviors of the ANNS graph traversing algorithm. CXL-ANNS is also aware of the architectural structures of the CXL interconnect network and allows different hardware components therein to simultaneously search for nearest neighbors in a collaborative manner. To improve the performance further, we relax the execution dependency in the kNN search and maximize the degree of search parallelism by fully utilizing all our hardware in the CXL network.

We summarize the main contribution of this work as follows:

- *Relationship-aware graph caching.* Since ANNS traverses a given graph from its entry-node [15, 48], we observe that the graph data accesses, associated with the innermost edge hops, account for most of the point accesses (Section 3.2). Inspired by this, we selectively locate the graph and feature vectors in different places of the CXL memory network. Specifically, CXL-ANNS allocates the node information closer to the entry node in the locally-attached DRAMs while placing the other datasets in the CXL memory pool.
- *Hiding the latency of CXL memory pool.* If it needs to traverse (uncached) outer nodes, CXL-ANNS prefetches the datasets of neighbors, most likely to be processed in the next step of kNN queries from the CXL memory pool. However, it is non-trivial to figure out which node will be the next to visit because of ANNS's procedural data processing dependency. We propose a simple foreseeing technique that exploits a unique graph traversing characteristic of ANNS and prefetches the next neighbor's dataset during the current kNN candidate update phase.
- *Collaborative kNN search design in CXL.* CXL-ANNS significantly reduces the time wasted for transferring the feature vectors back and forth by designing EP controllers to calculate distances. On the other hand, it utilizes the computation power of the CXL host for non-beneficial operations in processing data near memory (e.g., graph traverse and candidate update). This collaborative search includes an efficient design of RC-EP interfaces and a sharding method being aware of the hardware configurations of the CXL memory pool.
- *Dependency relaxation and scheduling.* The computation sequences of ANNS are all connected in a serial order, which makes them unfortunately dependent on execution. We examine all the activities of kNN query requests and classify them into urgent/deferrable subtasks.

²In this article, endpoint devices refer to devices other than CPU and switch that can request or complete CXL transaction. For example, a memory expander and an accelerator, connected to CXL network, are CXL endpoint devices.

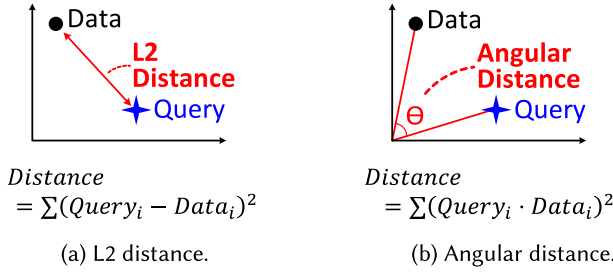


Fig. 2. Distance.

CXL-ANNS then relaxes the dependency of ANN computation sequences and schedules their subtasks at a finer granularity.

We validate all the functionalities of CXL-ANNS’s software and hardware (including the CXL memory pool) by prototyping them using Linux 5.15.36 and 16 nm FPGA, respectively. To explore the full design spaces of ANNS, we also implement the hardware-validated CXL-ANNS in gem5 [47] and perform full-system simulations using six billion-point datasets [58]. Our evaluation results show that CXL-ANNS exhibits 111.1× higher bandwidth (QPS) with 93.3% lower query latency, compared to the state-of-the-art billion-scale ANNS methods [30, 32, 56]. The latency and throughput behaviors of CXL-ANNS are even better than those of the oracle system (DRAM-only) by 68.0% and 3.8×, respectively.

2 BACKGROUND

2.1 Approximate Nearest Neighbor Search

The most accurate method to get kNN in a graph is to compare an input query vector with all data vectors in a brute-force manner [3, 16]. Obviously, this simple dense retrieval technique is impractical mainly due to its linear time complexity [30, 48]. In contrast, ANNS restricts the query vector to explore only a subset of neighbors that can be kNN with a high probability. To meet diverse accuracy and performance requirements, several ANNS algorithms such as tree-structure-based [51, 64], hashing-based [18, 27, 60], and quantization-based approaches [6, 17, 21, 32] have been proposed over the past decades. Among the various techniques, ANNS algorithms using graphs [15, 30, 48] are considered as the most promising solution, with great potential.³ This is because graph-based approaches can better describe neighbor relationships and traverse fewer points than the other approaches that operate in an Euclidean space [5, 14, 15, 41, 66].

Distance calculations. While there are various graph construction algorithms for ANNS [15, 30, 48], the goal of their query search algorithms is all the same or similar to each other; it is simply to find k number of neighbors in the target graph, which are expected to have the shortest distance from a given feature vector, called *query vector*. There are two most common methods to define such a distance between the query vector and neighbor’s feature vector (called *data vector*): (i) L2 (Euclidean) distance and (ii) angular distance. As shown in Figure 2, these methods map the nodes that we compare into a high-dimensional space using their own vector’s feature elements. Let us suppose that there are n numbers of features for each vector. Then, L2 and angular distances are calculated by $\sum_i (Query_i - Data_i)^2$ and $\sum_i (Query_i \cdot Data_i)$, respectively; where $Query_i$ and $Data_i$ are the i th features of a given query and data vectors, respectively ($i \leq n$).

³For the sake of the brevity, we use “graph-based approximate kNN methods” and “ANNS” interchangeably.

ALGORITHM 1: Best-first Search

```

Input: query,
k := number of nearest neighbor to find
Output: k nearest neighbor of query
/* Initialization (line 1~4) */
1 distance = calcDist(query, startNode)
2 CandidateArr = [{startNode, distance}]
3 curNode = startNode
/* Main iteration (line 4~9) */
4 while not candidateArr.allVisited() do
    /* Graph traversal (line 5~6) */
    5 CandidateArr.markVisited(curNode)
    6 neighbors = Graph.neighbors(curNode)
    /* Distance calculation (line 7) */
    7 distances = calcDist(query, neighbors)
    /* Candidate update (line 8~9) */
    8 CandidateArr.insert(neighbors, distances)
    9 curNode = CandidateArr.getNextNode()
10 return CandidateArr[: k]

```

The definitions of distances have been streamlined to cut down on computational time, deviating from the conventional definitions in a multi-dimensional vector space. This adjustment is grounded in the realization that ANNS employs these distances solely for relative kNN searches. In essence, ANNS omits certain operations that aren't essential for such comparisons. For instance, the presented L2 distance forgoes the square root step inherent in its standard definition, namely, $\sqrt{\sum_i (Query_i - Data_i)^2}$. This omission is rationalized by understanding that for relative comparisons, one can discern the outcome without undertaking this operation; for example, if $distance_a < distance_b$, then $\sqrt{distance_a} < \sqrt{distance_b}$.

In a similar vein, the discussed angular distance bypasses the calculation of the denominator in its conventional definition, which is $\sum_i (Query_i \cdot Data_i) / (\sqrt{\sum_i Query_i^2} \cdot \sqrt{\sum_i Data_i^2})$. Given that the distances are determined using a consistent query vector across varied neighbors, $\sqrt{\sum_i Query_i^2}$ is effectively a constant, rendering the division by it redundant for relative comparisons. Conversely, the division by $\sqrt{\sum_i Data_i^2}$ is addressed beforehand by adjusting the data vector elements to $NewData_i = Data_i / \sqrt{\sum_i Data_i^2}$ for every $i \leq n$. Such refinements expedite distance computations, particularly when leveraging multiple processing units for per-element tasks and subsequent aggregation.

Approximate kNN query search. Algorithm 1 explains the graph traversing method that most ANNS employs [15, 30, 48]. The method, **best-first search (BFS)** [23, 66], traverses from an *entry-node* (line ③) and moves to neighbors getting closer to the given query vector (lines ④~⑨). While the brute-force search explores a full space of the graph by systematically enumerating all the nodes, ANNS uses a preprocessed graph and visits a limited number of nodes for each hop. The graph is constructed (preprocessed) to have the entry-node that arrives all the nodes of its original graph within the minimum number of average edge hops; this preprocessed graph guarantees that

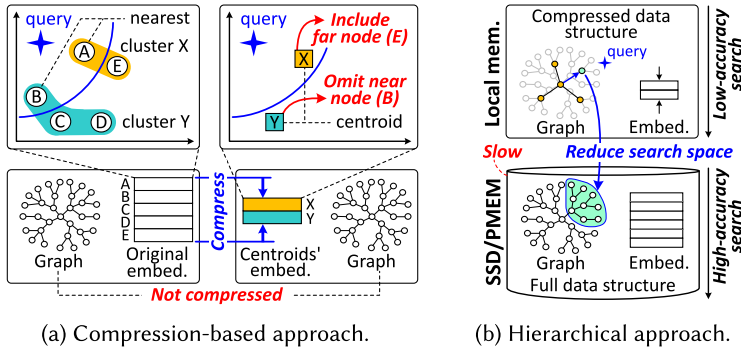


Fig. 3. Existing billion-scale ANNS methods.

there exists a path between the entry-node and any of the given nodes. To minimize the overhead of graph traversal, BFS employs a *candidate array* that includes the neighbors whose distances (from the query vector) are expected to be shorter than others. For each node visiting, BFS checks this candidate array and retrieves unvisited node from the array (line ⑥, ⑨). It then calculates the distances of the node's neighbors (line ⑦). This distance calculation retrieves the neighbors' vectors from the embedding table that manages the vectors for all the nodes in a contiguous memory space. Then, BFS updates the candidate array with the new information, neighbors, and distances (line ⑧). All these activities are iterated (line ④) until there is no unvisited node in the candidate array. BFS finally returns the k number of neighbors in the candidate array.

2.2 Towards Billion-scale ANNS

While ANNS can achieve good search speed and reasonable accuracy (as it only visits the nodes in the candidate array), it still requires maintaining all the original graph and vectors in its embedding table. This renders ANNS difficult to have billion-point graphs that exhibit high memory demands in many production-level services [15, 57]. To address this issue, there have been many studies proposed [6, 17, 19, 21, 30, 32, 56, 59], but we can classify them into two as shown in Figures 3(a) and 3(b).

Compression approaches. The methods proposed in [6, 17, 21, 32] aim at condensing the embedding table by compressing its vectors. As illustrated in Figure 3(a), these methods logically segment the provided graph into several sub-groups, termed *clusters*. For instance, nodes A and E are categorized into cluster X, while the rest fall under cluster Y. Within each cluster, the associated vectors are encoded into a singular, representative vector, termed a *centroid*, by averaging all cluster vectors. Subsequently, every vector in the embedding table is supplanted by its cluster ID. Nonetheless, this results in a challenge: distances are computed using the compressed centroid vectors instead of the original data vectors, leading to reduced search accuracy. For instance, even though node B is closer to the query vector, node E might still be chosen as one of the kNN.

Numerous strategies have been explored to bolster search precision. Product quantization [32] stands as an archetype for compression techniques. It divides the embedding vector into several sub-vectors and encodes each using distinct centroid sets. This division ensures that each centroid set more accurately represents its corresponding sub-vectors, thus improving search precision. Alternatively, ScaNN [21] specifically enhances precision when ANNS employs the angular distance metric. It uses an optimization algorithm that tailors the graph segmentation, penalizing angular distances between centroids and their affiliated neighbors. This diverges from the K-means clustering strategy used in product quantization, which exclusively caters to L2 distance [24]. However, a

limitation persists: search accuracy remains capped because the compression process necessitates representing numerous original data vectors with a finite set of centroids. This limitation will be further dissected in Section 3.1.

Another drawback of these compression techniques is the modest reduction they achieve in graph dataset sizes. Since only the embedding table is quantized, their expansive billion-point graph data does not benefit from compression. In some cases, the data might even slightly expand due to the inclusion of shortcuts in the original graph.

Hierarchical approaches. These approaches [19, 30, 56, 59] store all the graphs and vectors (embedding table) to the underlying SSD/PMEM (Figure 3(b)). Since SSD/PMEM is practically slower than DRAM by many orders of magnitude, these methods process kNN queries in two separate phases: (i) *low-accuracy search* and (ii) *high-accuracy search*. The former only refers to compressed or simplified datasets, similar to the datasets that the compression approaches use. The low-accuracy search quickly finds out one or more nearest neighbor candidates (without a storage access) thereby reducing the search space that the latter needs to process. Once it has been completed, the high-accuracy search refers to the original datasets associated with the candidates and processes the actual kNN queries. For example, DiskANN [30]’s low-accuracy search finds the kNN with a larger k using the compressed embedding table in DRAM.

DiskANN utilizes the graph in SSD/PMEM for a search with lower accuracy. In this process, it simultaneously accesses the graph structures adjacent to several vertices, which enhances the performance of graph traversal. Subsequently, every node that was visited during the low-accuracy search is designated as a candidate. The high-accuracy search phase then re-evaluates and re-orders the kNN candidates by referencing their actual vectors present in SSD/PMEM.

In contrast, HM-ANN [56] refines the target graph by integrating multiple shortcuts that span across several edge hops. During its low-accuracy search, HM-ANN identifies a candidate closer to the specified query vector using this streamlined graph. After pinpointing this candidate, the high-accuracy search phase determines its kNN by executing the best-first search algorithm once more, leveraging both the graph and data vectors stored in SSD/PMEM. Notably, during this process, the candidate node is used as the entry point. As a result, the search algorithm is anticipated to conclude within a limited number of iterations since the entry node is already in proximity to the query vector. Although the low-accuracy search reduces the number of nodes to be examined in the high-accuracy search, the latter still notably impacts the search latency due to the inherent long latency of SSD/PMEM. This issue will be further discussed in Section 3.1.

2.3 Compute Express Link for Memory Pool

CXL is an open standard interconnect which can expand memory over the existing PCIe physical layers in a scalable option [11, 40, 50]. As shown in Figure 4(a), CXL consists of three sub-protocols: (i) *CXL.io*, (ii) *CXL.cache*, and (iii) *CXL.mem*. Based on which sub-protocols are used for the main communication, CXL EPs can be classified as Types.

Sub-protocols and endpoint types. *CXL.io* is basically the same as the PCIe standard, which is aimed at enumerating the underlying EPs and performing transaction controls. It is thus used for all the CXL types of EPs to be interconnected to the CXL CPU’s RC through PCIe. On the other hand, *CXL.cache* is for an underlying EP to make its states coherent with those of a CXL host CPU, whereas *CXL.mem* supports simple memory operations (load/store) over PCIe. Type 1 is considered by a co-processor or accelerator that does not have memory exposed to CXL RC while Type 2 employs internal memory, accessible from CXL RC. Thus, Type 1 only uses *CXL.cache* (in addition to *CXL.io*), but Type 2 needs to use both *CXL.cache* and *CXL.mem*. A potential example of Type 1 and 2 can be FPGAs and GPUs, respectively. On the other hand, Type 3 only uses *CXL.mem*

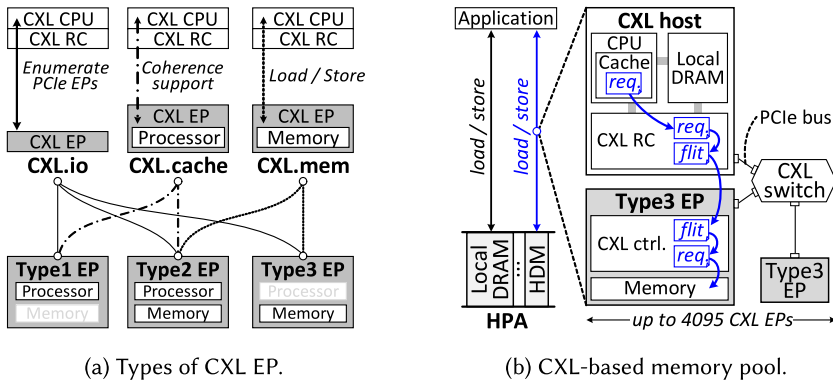


Fig. 4. CXL's sub-protocols and endpoint types.

(read/write), which means that there is no interface for a device-side compute unit to update its calculation results to CXL CPU's RC and/or get a non-memory request from the RC.

CXL endpoint disaggregation. Figure 4(b) shows how we can disaggregate DRAM from host resources using CXL EPs, in particular, Type 3; we will discuss why Type 3 is the best device type for the design of CXL-ANNS, shortly. Type 3’s internal memory is exposed as a *host-managed device memory* (HDM), which can be mapped to the CXL CPU’s **host physical address (HPA)** in the system memory just like DRAM. Therefore, applications running on the CXL CPU can access HDM (EP’s internal memory) through conventional memory instructions (loads/stores). Thanks to this characteristic, HDM requests are treated as traditional memory requests in CXL CPU’s memory hierarchy; the requests are first cached in CPU cache(s). Once its cache controller evicts a line associated with the address space of HDM, the request goes through to the system’s CXL RC. RC then converts one or more memory requests into a CXL packet (called *flit*) that can deal with a request or response of CXL.mem/CXL.cache. RC passes the flit to the target EP using CXL.mem’s read or write interfaces. The destination EP’s PCIe and CXL controllers take the flit over, convert it to one or more memory requests, and serve the request with the EP’s internal memory (HDM).

Scaling system using CXL switch. To increase memory capacity, the designated CXL network can incorporate one or several switches, each boasting multiple ports. In its most basic configuration, a sole switch can be used to link numerous EPs to a host. These EPs are connected to the switch’s downstream ports, whereas the host is linked to its upstream port. When the host sends a CXL flit to the switch’s upstream port, the switch directs the flit to the appropriate downstream port by consulting its internal routing table. This table associates an address range tied to an EP’s HDM with its corresponding downstream port, set during initialization by the host. Consequently, the switch can determine the route for the CXL flit by checking its destination address and then consulting the routing table using that address to identify the correct downstream port.

Using a single switch does facilitate the connection of multiple EPs to one host. However, its scalability is constrained by the finite number of ports on the switch. To address this, CXL 3.0 introduced a multi-level switch feature, which significantly amplifies the number of EPs that can be linked to a host.

Type consideration for scaling-out. This switch-based network configuration allows an RC to employ many EPs, but only for Type 3. This is because CXL.cache uses virtual addresses for its cache coherence management unlike CXL.mem. As the virtual addresses (brought by CXL flits) are not directly matched with the physical address of each underlying EP’s HDM, the CXL switches cannot understand where the exact destination is.

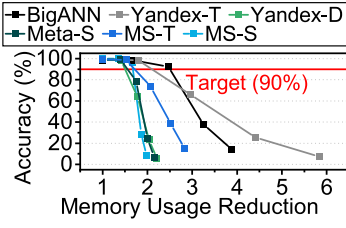


Fig. 5. Accuracy.

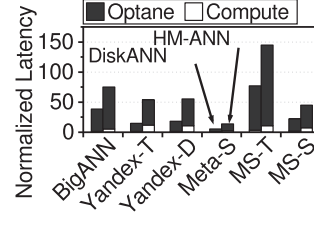


Fig. 6. Latency.

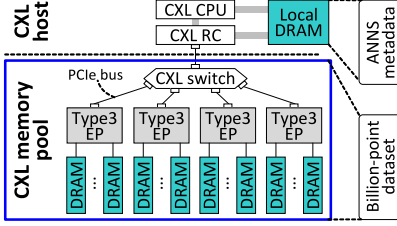


Fig. 7. CXL baseline.

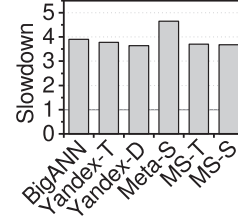


Fig. 8. CXL slowdown.

3 A HIGH-LEVEL VIEWPOINT OF CXL-ANNS

3.1 Challenge Analysis of Billion-scale ANNS

Memory expansion with compression. While compression methods allow us to have larger datasets, it is not scalable since their quantized data significantly degrades the kNN search accuracy. Figure 5 analyzes the search accuracy of billion-point ANNS that uses the quantization-based compression described in Section 2.2. In this analysis, it reduces the embedding table by $2 \times \sim 16 \times$. We use six billion-point datasets from [58]; the details of these datasets and evaluation environment are the same as what we used in Section 6.1. As the density of the quantized data vectors varies across different datasets, the compression method exhibits different search accuracies. While the search accuracies are in a reasonable range to service with low compression rates, they significantly drop as the compression rate of the dataset increases. It cannot even reach the threshold accuracy that ANNS needs to support (90%, recommended by [58]) after having 45.8% less data than the original. This unfortunately makes the compression impractical for billion-scale ANNS at high accuracy.

Hierarchical data processing. Hierarchical approaches can overcome this low accuracy issue by adding one more search step to re-rank the results of kNN search. This high-accuracy search however increases the search latency significantly as it eventually requires traversing the storage-side graph and accessing the corresponding data vectors (in storage) entirely. Figure 6 shows the latency behaviors of hierarchical approaches, DiskANN [30] and HM-ANN [56]. In this test, we use 480 GB Optane PMEM [29] for DiskANN/HM-ANN and compare their performance with the performance of an oracle ANNS that has DRAM-only (with unlimited storage capacity). One can observe from this figure that the storage accesses of the high-accuracy search account for 87.6% of the total kNN query latency, which makes the search latency of DiskANN and HM-ANN worse than that of the oracle ANNS by $29.4 \times$ and $64.6 \times$, respectively, on average.

CXL-augmented ANNS. To avoid the accuracy drop and performance depletion, this work advocates to directly have billion-point datasets in a scalable memory pool, disaggregated using CXL. Figure 7 shows our baseline architecture that consists of a CXL CPU, a CXL switch, and four 1TB Type 3 EPs that we prototype (Section 6.1). We locate all the billion-point graphs and corresponding

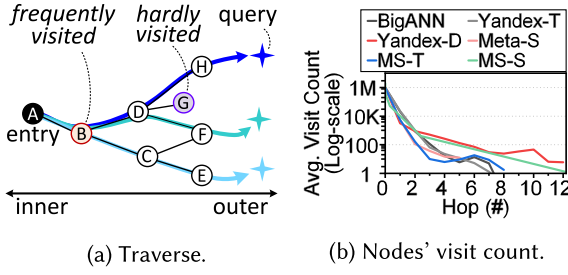


Fig. 9. Graph traverse.

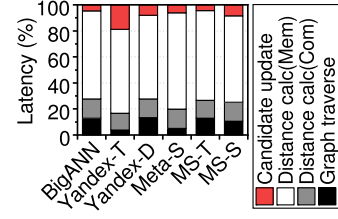


Fig. 10. End-to-end latency breakdown analysis.

vectors to the underlying Type 3 EPs (memory pool) while having ANNS metadata (e.g., candidate array) in the local DRAM. This baseline allows ANNS to access the billion-point datasets on the remote-side memory pool just like conventional DRAMs thanks to CXL's instruction-level compatibility. Nevertheless, it is not yet an appropriate option for practical billion-scale ANNS due to CXL's architectural characteristics that exhibit lower performance than the local DRAM.

To be precise, we compare the kNN search latency of the baseline with the oracle ANNS, and the results are shown in Figure 8. In this analysis, we normalize the latency of the baseline to that of the oracle for better understanding. Even though our baseline does not show severe performance depletion like what DiskANN/HM-ANNS suffer from, it exhibits $3.9\times$ slower search latency than the oracle, on average. This is because all the memory accesses associated with HDM(s) insist the host RC convert them to a CXL flit and revert the flit to memory requests at the EP-side. The corresponding responses also requires this *memory-to-flit* conversion in a reverse order thereby exhibiting the long latency for graph/vector accesses. Note that this $3.6\sim 4.6\times$ performance degradation is not acceptable in many production-level ANNS applications such as recommendation systems [2] or search engines [13].

3.2 Design Consideration and Motivation

The main goal of this work is to make the CXL-augmented kNN search faster than in-memory ANNS services working only with locally-attached DRAMs (cf. CXL-ANNS vs. Oracle as shown in Figure 8). To achieve this goal, we propose CXL-ANNS, a software-hardware collaborative approach, which considers the following three motivations: (i) node-level relationship, (ii) distance calculation, and (iii) vector reduction.

Node-level relationship. While there are diverse graph structures [15, 30, 48] for the best-first search traverses (cf. Algorithm 1), all of the graphs starts their traversals from a unique, single entry-node as described in Section 2.1. This implies that the graph traverse of ANNS visits the nodes closer to the entry-node much more frequently. For example, as shown in Figure 9(a), the node B is always accessed to serve a given set of kNN queries targeting other nodes listed in the graph branch while the node G is difficult to visit. To be precise, we examine the average count to visit nodes in all the billion-point graphs that this work evaluates when there are a million kNN query requests. The results are shown in Figure 9(b). One can observe from this analysis that the nodes most frequently accessed during the 1M kNN searches reside in the 2~3 edge hops. By appreciating this node-level relationship, we will locate the graph and vector data regarding innermost nodes (from the entry-node) to locally-attached DRAMs while allocating all the others to the underlying CXL EPs.

Distance calculation. To analyze the critical path of billion-point ANNS, we decompose the end-to-end kNN search task into four different sub-tasks, (i) candidate update, (ii) memory access and

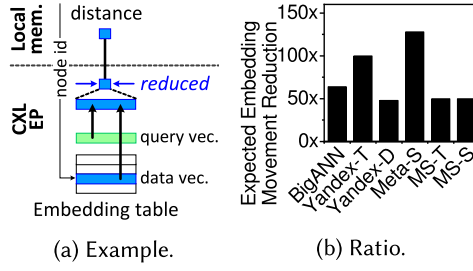


Fig. 11. Data reduction.

(iii) computing fractions of distance calculation, and (iv) graph traverse. We then measure the latency of each sub-tasks on use in-memory, oracle system, which are shown in Figure 10. As can be seen from the figure, ANNS distance calculation significantly contributes to the total execution time, constituting an average of 81.8%. This observation stands in contrast to the widely held belief that graph traversal is among the most resource-intensive operations [1, 12, 55]. The underlying reason for this discrepancy is that distance calculation necessitates intensive embedding table lookups to determine the data vectors of all nodes visited by ANNS. Notably, while these lookup operations have the same frequency and pattern as graph traversal, the length of the data vectors employed by ANNS is 2.0 $\times$ greater than that of the graph data due to their high dimensionality. Importantly, although distance calculation exhibits considerable latency, it does not require substantial computational resources, thus making it a good candidate for acceleration using straightforward hardware solutions.

Reducing data vector transfers. We can take the overhead brought by distance calculations off the critical path in the kNN search by bringing only the distance that ANNS needs to check for each iteration its algorithm visits. As shown in Figure 11(a), let's suppose that CXL EPs can compute a distance between a given query vector and data vectors that ANNS is in visit. Since ANNS needs the distance, a simple scalar value, instead of all the full features of each data vector, the amount of data that the underlying EPs transfer can be reduced as many as each vector's dimensional degrees. Figure 11(b) analyzes how much we can reduce the vector transfers during services of the 1M kNN queries. While the vector dimensions of each dataset varies (96~256), we can reduce the amount of data to load from the EPs by 73.3 $\times$, on average.

3.3 Collaborative Approach Overview

Motivated by the aforementioned observations, CXL-ANNS first caches datasets considering a given graph's inter-node relationship and performs ANNS algorithm-aware CXL prefetches (Section 4.1). This makes the performance of a naive CXL-augmented kNN search comparable with that of the oracle ANNS. To go beyond, CXL-ANNS reduces the vector transferring latency significantly by letting the underlying EPs to calculate all the ANNS distances near memory (Section 5.1). As this near-data processing is achieved in a collaborative manner between EP controllers and RC-side ANNS algorithm handler, the performance can be limited by the kNN query service sequences. CXL-ANNS thus schedules kNN search activities in a fine-grained manner by relaxing their execution dependency (Section 5.3). Putting all together, CXL-ANNS is designed for offering high-performance even better than the oracle ANNS without an accuracy loss.

Figure 12 shows the high-level viewpoint of our CXL-ANNS architecture, which mainly consists of (i) RC-side software stack and (ii) EP-side data processing hardware stack.

RC-side software stack. This RC-side software stack is composed of (i) query scheduler, (ii) pool manager, and (iii) kernel driver. At the top of CXL-ANNS, the query scheduler handles all kNN

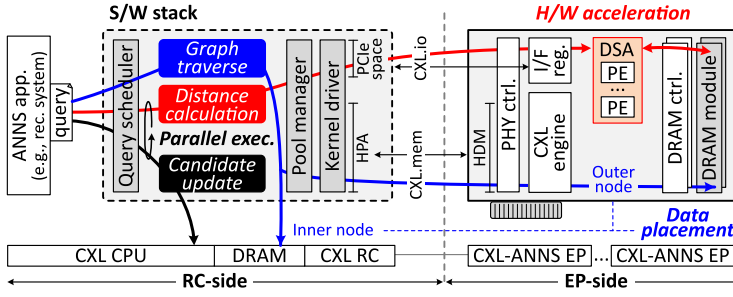


Fig. 12. Overview.

searches requested from its applications such as recommendation systems. It splits each query into three subtasks (graph traverse, distance calculation, and candidate update) and assigns them in different places. Specifically, the graph traverse and candidate update subtasks are performed at the CXL CPU side whereas the scheduler allocates the distance calculation to the underlying EP by collaborating with the underlying pool manager. The pool manager handles CXL's HPA for the graph and data vectors by considering edge hop counts, such that it can differentiate graph accesses based on the node-level relationship. Lastly, the kernel driver manages the underlying EPs and their address spaces; it enumerates the EPs and maps their HDMs into the system memory's HPA that the pool manager uses. Since all memory requests for HPA are cached at the CXL CPU, the driver maps EP-side interface registers to RC's PCIe address space using CXL.io instead of CXL.mem. Note that, as the PCIe spaces where the memory-mapped registers exist is in non-cacheable area, the underlying EP can immediately recognize what the host-side application lets the EPs know.

EP-side hardware stack. EP-side hardware stack includes a **domain specific accelerator (DSA)** for distance calculation in addition to all essential hardware components to build a CXL-based memory expander. At the front of our EPs, a physical layer (PHY) controller and CXL engine are implemented, which are responsible for the PCIe/CXL communication control and flit-to-memory request conversion, respectively. The converted memory request is forwarded to the underlying memory controller that connects multiple DRAM modules at its backend; in our prototype, an EP has four memory controllers, each having a DIMM channel that has 256 GB DRAM modules. On the other hand, the DSA is located between the CXL engine and memory controllers. It can read data vectors using the memory controllers while checking up the operation commands through CXL engine's interface registers. These interface registers are mapped to the host non-cacheable PCIe space such that all the commands that the host writes can be immediately visible to DSA. DSA calculates the approximate distance for multiple data vectors using multiple **processing elements (PEs)**, each having simple arithmetic units such as adder/subtractor and multiplier.

4 SOFTWARE STACK DESIGN AND IMPLEMENTATION

From the memory pool management viewpoint, we have to consider two different system aspects: (i) graph structuring technique for the local memory and (ii) efficient space mapping method between HDM and graph. We will explain the design and implementation details of each method in this section.

4.1 Local Caching for Graph

Graph construction for local caching. While the pool manager allocates most graph data and all data vectors to the underlying CXL memory pool, it caches the nodes, expected to be most frequently accessed, in local DRAMs as much as the system memory capacity can accommodate.

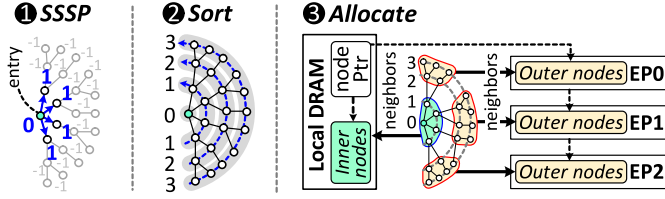


Fig. 13. Data placement.

To this end, the pool manager considers how many edge hops exist from the fixed entry-node to each node for its relationship-aware graph cache.

Figure 13 explains how the pool manager allocates the nodes in a given graph to different places (local memory vs. CXL memory pool). To consider the number of edge hops, the pool manager calculates the number of edge hops for all nodes at graph construction time. Specifically, the calculation leverages a **single source shortest path (SSSP)** algorithm [61, 74]; it first lets all the nodes in the graph have a negative hop count (e.g., -1). Starting from the entry-node, the pool manager checks all the nodes in one edge hop and increases its hop count. It visits each of the nodes and iterates this process for them until there is no node to visit in a breadth-first search manner. Once each node has its own hop count, the pool manager sorts them based on the hop count in an ascending order and allocates the nodes from the top (having the smallest hop count) to local DRAMs as many as it can. The available size of the local DRAMs can be simply estimated by referring to system configuration variables (`sysconf()`) of the total number of pages (`_SC_AVPHYS_PAGES`) and the size of each page (`_SC_PAGESIZE`).

Notably, in this study, we designed the pool manager as a user-level library. This allows the pool manager to leverage multiple threads for executing SSSP thereby reducing the construction time to a minimum. Once the construction is done, the threads are terminated to make sure they do not consume CPU resources when a query is given.

4.2 Data Placement on the CXL Memory Pool

Preparing CXL for user-level memory. When mapping HDM to the system memory's HPA, CXL CPU should be capable of recognizing different HDMs and their size whereas each EP needs to know where its HDM is assigned in HPA. As shown in Figure 14, our kernel driver checks PCIe configuration space and figures out CXL devices at the PCIe enumeration time. The driver then checks RC information from the system's data structure describing the hardware components that show where the CXL HPA begins (base), such as device tree [44] or ACPI [62]. From the base, our kernel driver allocates each HDM as much as it defines in a contiguous space. It lets the underlying EPs know where each of corresponding HDM is mapped in HPA, such that they can convert the address of memory requests (HPA) to its original HDM address. Once all the HDMs are successfully mapped to HPA, the pool manager allocates each HDM to different places of user-level virtual address space that the query scheduler operates on. This memory-mapped HDM, called *CXL arena*, guarantees per-arena continuous memory space and allows the pool manager to distinguish different EPs at the user-level.

Pool management for vectors/graph. While CXL arenas directly expose the underlying HDMs of CXL EPs to user-level space, it should be well managed to accommodate all the billion-point datasets appreciating their memory usage behaviors. The pool manager considers two aspects of the datasets; the data vectors (i.e., embedding table) should be located in a substantially large and consecutive memory space while the graph structure requires taking many neighbor lists with variable length (16 B~1 KB). The pool manager employs stack-like and buddy-like memory

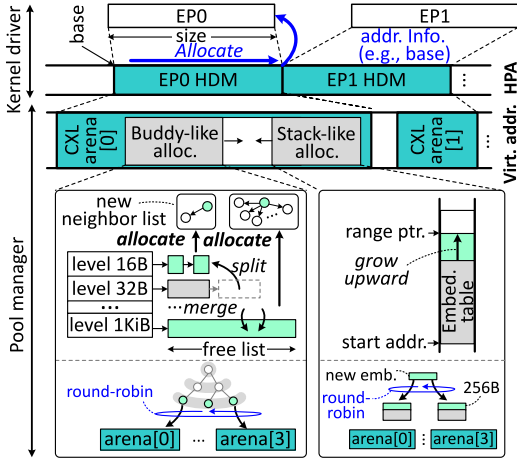


Fig. 14. Memory management.

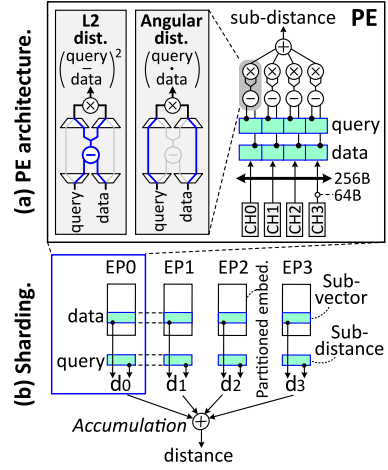


Fig. 15. Distance calculation.

allocators, which grow upward and downward in each CXL arena, respectively. The former allocator has a range pointer and manages memory for the embedding table, similar to stack. The pool manager allocates the data vectors across multiple CXL arenas in a round-robin manner by considering the underlying EP architecture. This vector sharding method will be explained in Section 5.1. In contrast, the buddy-like allocator employs a level pointer, each level consisting of a linked list, which connects data chunks with different sizes (from 16 B to 1 KB). Like Linux buddy memory manager [35], it allocates the CXL memory spaces as much as each neighbor list exactly requires and merge/split the chunk(s) based on the workload behaviors. To make each EP balanced, the pool manager allocates the neighbor lists for each hop in a round-robin manner across different CXL arenas.

5 COLLABORATIVE QUERY SERVICE ACCELERATION

5.1 Accelerating Distance Calculation

Distance computing in EP. As shown in Figure 15(a), a PE of DSA has an arithmetic logic tree connecting a multiplier and subtractor at each terminal for element-wise operations. Depending on how the dataset's features are encoded, the query and data vectors are routed differently to the two units as input. If the features are encoded for the Euclidean space, the vectors are supplied to the subtractor for L2 distance calculation. Otherwise, the multiplexing logics directly deliver the input vectors to the multiplier by bypassing the subtractor such that it can calculate the angular distance. Each terminal simultaneously calculates individual elements of the approximate distance, and the results are accumulated by going through the arithmetic logic tree network from the terminal to its root. In addition, each PE's terminal reads data from all four different DIMM channels in parallel, thus maximizing the EP's backend DRAM bandwidth.

Vector sharding. Even though each EP has many PEs (10 in our prototype), if we locate the embedding table from the start address of an EP in a consecutive order, EP's backend DRAM bandwidth can be bottleneck in our design. This is because each feature vector in the embedding table is encoded by high dimensional information (~ 256 dimensions, taking around 1 KB). To address this, our pool manager shards the embedding table in a column wise and stores different parts of the table across the different EPs. As shown in Figure 15b, this *vector sharding* splits each vector into multiple sub-vectors based on each EP's I/O granularity (256 B). Each EP simultaneously

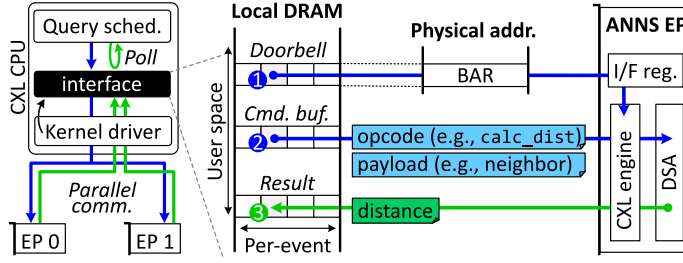


Fig. 16. Interface.

computes its sub-distance from the split data vector that the EP accommodates. Later, the CXL CPU accumulates the sub-distances to get the final distance value. Note that, since the L2 and angular distances are calculated by accumulating the output of element-wise operations, the final distance is the same as the results of the sub-distance accumulation using vector sharding.

Interfacing with EP-level acceleration. Figure 16 shows how the interface registers are managed to let the underlying EPs compute a distance where the data vectors exist. There are two considerations for the interface design and implementation. First, multiple EPs perform the distance calculation for the same neighbors in parallel thanks to vector sharding. While the neighbor list contains many node ids (≤ 200), it is thus shared by the underlying EPs. Second, handling interface registers using CXL.io is an expensive operation as CPU should be involved in all the data copies. Considering these two, the interface registers handle only the event of command arrivals, called *doorbell*, whereas each EP's CXL engine pulls the corresponding operation type and neighbor list from the CPU-side local DRAM (called a command buffer) in an active manner. This method can save time for the CPU to move the neighbor list to each EP's interface registers one by one as the CXL engine brings all the information if there is any doorbell update. The CXL engine also pushes results of distance calculation to the local DRAM such that the RC-side software directly accesses the results without an access of the underlying CXL memory pool. Note that all these communication buffers and registers are directly mapped to the user-level virtual addresses in our design such that we can minimize the number of context switches between user and kernel mode.

5.2 Prefetching for CXL Memory Pool

Figure 17(a) shows our baseline of collaborative query service acceleration, which lets EPs compute sub-distances while ANNS's graph traverse and candidate update (including sub-distance accumulation) are handled at the CPU-side. This scheduling pattern is iterated until there are no kNN candidates to visit further (Algorithm 1). A challenge of this baseline approach is traversing graph can be started once the all node information is ready at the CPU-side. While local caching of our pool manager addresses this, it yet shows a limited performance. It is required to go through the CXL memory pool to get nodes, which does not sit in the innermost edge hops. As its latency to access the underlying memory pool is long, the graph traverse can be postponed comparably. To this end, our query scheduler prefetches the graph information earlier than the actual traverse subtask needs, as shown in Figure 17.

While prefetching can mitigate the extended latency arising from CXL memory pool accesses, it's not straightforward. This is because prefetching demands prior knowledge of the nodes the subsequent iteration of the ANNS algorithm will access. Our query scheduler predicts these nodes and fetches their neighboring data, basing this on the candidate array. This approach stems from an observation we made. As depicted in Figure 18, when examining the nodes accessed during the next iteration's graph traversal across all our test datasets, we noted that a significant 82.3%

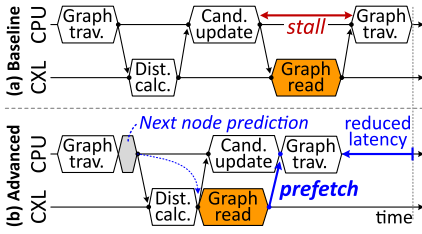


Fig. 17. Prefetching.

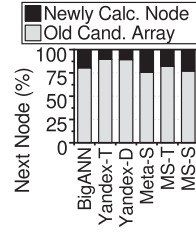


Fig. 18. Next node's source.

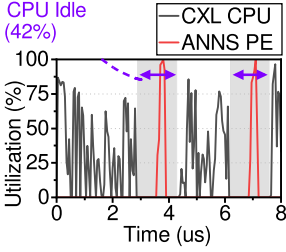


Fig. 19. Resource utilization.

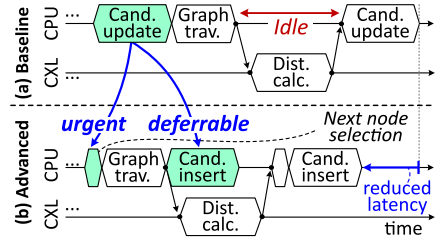


Fig. 20. Query scheduling.

of the nodes accessed originated from the candidate array. This is notable, especially considering this array's data hadn't been updated for the upcoming step.

Given this insight, our query scheduler preemptively fetches nodes from the candidate array, without awaiting its update. In the best-first search algorithm, the next node to explore is the nearest one among those yet to be visited. To efficiently identify this nearest node, our query scheduler arranges nodes within the candidate array in ascending order by distance. Additionally, it maintains a boolean flag for each node, signifying its visitation status. To prefetch the upcoming node for exploration, the scheduler starts scanning the array from the beginning, looking for an unvisited node. Upon identifying one, it prefetches this node utilizing the `__builtin_prefetch()` function from gcc. Due to the instruction-level compatibility offered by CXL, this built-in function can be applied to the CXL memory space without any modifications.

5.3 Fine-Granular Query Scheduling

Our collaborative search query acceleration can reduce the amount of data to transfer significantly and successfully hide the long latency imposed by the CXL memory pool. However, computing kNN search in different places makes the RC-side ANNS subtasks pending until EPs complete their distance calculation. Figure 19 shows how much the RC-side subtasks (CXL CPU) stay idle, waiting for the distance results. In this evaluation, we use Yandex-D as a representative of the datasets, and its time series are analyzed for the time visiting only first two nodes for their neighbor search. The CXL CPU performs nothing while EPs calculate the distances, which take 42% of the total execution time for processing those two nodes. This idle time cannot be easily removed as candidate update cannot be processed without having their distance.

To address this, our query scheduler relaxes the execution dependency on the candidate update and separates such an update into urgent and deferrable procedures. Specifically, the candidate update consists of (i) inserting (updating) the array with the candidates, (ii) sorting kNN candidates based on their distance, and (iii) node selection to visit. The node selection is an important process because the following graph traverse requires knowing the nodes to visit (urgent). However, sorting/inserting kNN candidates maintain the k numbers of neighbors in the candidate array,



Fig. 21. Prototype.

Table 1. Simulation Setup

CPU	40 O3 cores, ARM v8, 3.6GHz L1/L2 \$: 64KiB/2MiB per core
Local memory	128GiB, DDR4-3200
CXL memory pool	1 CXL switch 256GiB/device, DDR4-3200
Storage	4× Intel Optane 900P 480 GB
CXL-ANNS	1 GHz, 5 ANNS PE/device, 2 distance calc. unit/PE

Table 2. Workloads

Dataset	Dist.	Num. vecs.	Emb. dim.	Avg. num. neighbors	Candidate arr. size			Num. devices.
					k = 1	k = 5	k = 10	
BigANN	L2	1B	128	31.6	30	75	150	4
Yandex-T	Ang.	1B	200	29.0	440	900	2500	4
Yandex-D	L2	1B	96	66.9	300	700	1700	4
Meta-S	L2	1B	256	190	1200	2800	5600	8
MS-T	L2	1B	100	43.1	60	130	250	4
MS-S	L2	1B	100	87.4	580	1000	2000	4

which are not to be done immediately. Thus, as shown in Figure 20, the query scheduler performs the node selection before the graph traverse, but it executes the deferrable operations during the distance calculation time by delaying them at in a fine-granular manner.

6 EVALUATION

6.1 Evaluation Setup

Prototype and Methodology. Given the lack of a publicly available, fully functional CXL system, we constructed and validated the CXL-ANNS software and hardware in an operational real system (Figure 21). This hardware prototype is based on a 16 nm FPGA. To develop our CXL CPU prototype, we adapted the RISC-V CPU [8]. The prototype integrates 4 ANNS EPs, each equipped with four memory controllers, linked to the CXL CPU via a CXL switch. The system’s software for the prototype, including the kernel driver, is compatible with Linux 5.15.36. For the ANNS execution, we adjusted Meta’s open ANNS library, FAISS v1.7.2 [31].

Unfortunately, the prototype system does not offer the flexibility needed to explore various ANNS design spaces. As a remedy, we also established a hardware-validated full-system simulator that represents CXL-ANNS, which was utilized for evaluation. The simulator is built on top of gem5 simulator [47] and employs a detailed timing model that replicates the behavior of our hardware prototype. For example, it models the memory-to-flit conversion time for a CXL request by using a set of events that describes the delays from CXL RC, CXL switch, EP-side CXL engine, and PCIe physical layer. All the delays are configured to match the operational cycles extracted from our hardware prototype and are cross-validated with our real system at the cycle level.

We conducted simulation-based studies in this evaluation, the system details of which are outlined in Table 1. Notably, the system emulates the server utilized in Meta’s production environment [22]. Although our system by default uses 4 EPs, our system increases their count for specific workloads (e.g., Meta-S) that necessitate larger memory spaces (more than 2 TB) compared to others.

Workloads. We use billion-scale ANNS datasets from BigANN benchmark [58], a public ANNS benchmark that multiple companies (e.g., Microsoft, Meta) participate in. Their important characteristics are summarized in Table 2. In addition, since ANNS-based services often need different number of nearest neighbors [15, 70], we evaluated our system on the various k (e.g., 1, 5, 10). We generated the graph for ANNS by using state-of-the-art algorithm, NSG, employed by production search services in Alibaba [15]. Since the accuracy and performance of BFS can vary on the size of the candidate array, we only show the performance behavior of our system when its accuracy is 90%, as recommended by the BigANN benchmark. The accuracy is defined as $recall@k$; the ratio of the exact k number of neighbors that are included in the k number of output nearest neighbors of ANNS.

Configurations. We compare CXL-ANNS with 3 state-of-the-art large-scale ANNS systems. For the compression approach, we use a representative algorithm, product quantization [32] (**Comp**). It compresses the data vector by replacing the vector with the centroid of its closest cluster (see Section 2.2). For the hierarchical approach, we use DiskANN [30] (**Hr-D**) and HM-ANN [56] (**Hr-H**) for the evaluation. The two methods employ compressed embedding table and simplified graphs to

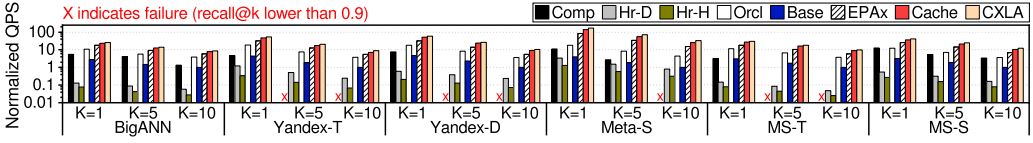


Fig. 22. Throughput (queries per second).

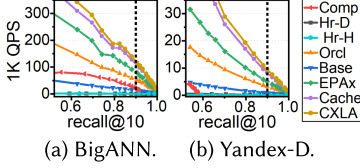


Fig. 23. Recall-QPS curve.

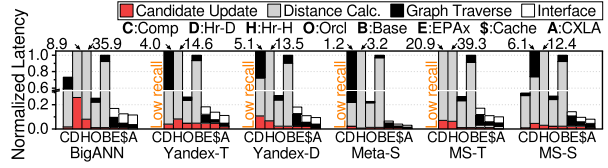
Fig. 24. Single query latency ($k = 10$).

Table 3. Latency

Dataset	BigANN	Yandex-T	Yandex-D	Meta-S	MS-T	MS-S
Base	3.0	66.0	55.7	1,121.2	6.0	107.2
CXL-ANNS	0.3	7.4	5.3	34.2	0.6	8.6

*unit: ms.

reduce the number of SSD/PMEM accesses, respectively. For fair comparison, we use the same storage device, Intel Optane [29], for both Hr-D/H. For CXL-ANNS, we evaluated its multiple variants to distinguish the effect of each method we propose. Specifically, **Base** places the graph and embedding table in CXL memory pool and lets CXL CPU execute the subtasks of ANNS. Compared to Base, **EPax** performs distance calculation by using DSA inside the ANNS EP. Compared to EPax, **Cache** employs relationship-aware graph caching and prefetching. Lastly, **CXLA** employs all the methods we propose, including fine-granular query scheduling. In addition, we compare oracle system (**Orcl**) that uses unlimited local DRAM. We will show that CXL-ANNS makes the CXL-augmented kNN search faster than Orcl.

6.2 Overall Performance

We first compare the throughput and latency of various systems we evaluated. We measured the systems' throughput by counting the number of processed queries per second (QPS, in short). Figure 22 shows the QPS of all k s, while Figure 24 digs the performance behavior deeper for $k = 10$ by breaking down the latency. We chose $k = 10$ following the guide from BigANN benchmark. The performance behavior for $k = 1, 5$ are largely same with when $k = 10$. For both figures, we normalized the values by that of **Base** when $k = 10$. The original latencies are summarized in Table 3.

As shown in Figure 22, the QPS gets lower when the k increases for all the systems we tested. This is because, the BFS visits more nodes to find more nearest neighbors. On the other hand, while **Comp** exhibits comparable QPS to **Orcl**, it fails to reach the target recall@ k (0.9) for 7 workloads. This is because **Comp** cannot calculate the exact distance since it replaces the original vector with the centroid of a cluster nearby. This can also be observed in Figure 23. The figure shows the accuracy and QPS when we vary the size of candidate array for two representative workloads. BigANN represents the workloads that **Comp** does reach the target recall, while Yandex-D represents the opposite. We can see that **Comp** converges at low recall@10 of 0.92 and 0.58, respectively, while other systems reach the maximum recall@10.

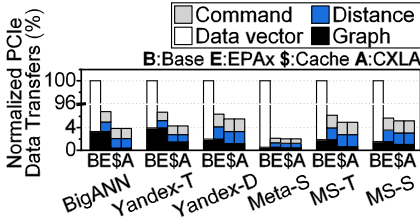


Fig. 25. Data transfer.

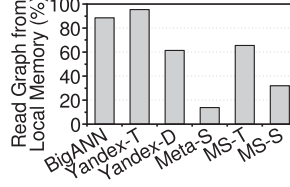


Fig. 26. Local caching.

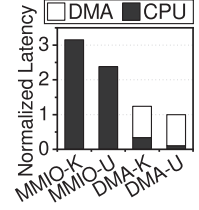


Fig. 27. Interface.

In contrast, hierarchical approaches (Hr-D/H) reaches the target recall@k for all the workloads we tested, by re-ranking the search result. However, they suffer from the long latency of underlying SSD/PMEM while accessing their uncompressed graph and embedding table. Such long latency significantly depletes the QPS of Hr-D and Hr-H by 35.9 $\times$ and 77.6 $\times$ compared to Orcl, respectively. Consider Figure 24 to better understand; Since Hr-D only calculates the distance for the limited number of nodes in the candidate array, it exhibits 20.1 $\times$ shorter distance calculation time compared to Hr-H, which starts a new BFS on original graph stored in SSD/PMEM. However, Hr-D's graph traverse takes longer time than that of Hr-H by 16.6 $\times$. This is because, Hr-D accesses the original graph in SSD/PMEM for both low/high-accuracy search while Hr-H accesses the original graph only for their high-accuracy search.

As shown in Figure 22, Base does not suffer from accuracy drop or the performance depletion of Hr-D/H since it employs a scalable memory pool that CXL offers. Therefore, it significantly improves the QPS by 9.4 $\times$ and 20.3 $\times$, compared to Hr-D/H, respectively. However, Base still exhibits 3.9 $\times$ lower throughput than Orcl. This is because Base experiences the long latency of memory-to-flit conversion while accessing the graph/embedding in CXL memory pool. Such conversion makes Base's graph traverse and distance calculation longer by 2.6 $\times$ and 4.3 $\times$, respectively, compared to Orcl.

Compared to Base, EPAX significantly diminishes the distance calculation time by a factor of 119.4 $\times$, achieved by reducing data vector transfer through the acceleration of distance calculation within the EPs. While this EP-level acceleration introduces an interface overhead, this overhead only represents 5.4% of the Base's distance calculation latency. Hence, EPAX reduces the query latency by 7.5 $\times$ on average, relative to Base. It's important to highlight that EPAX's latency is 1.9 $\times$ lower than Orcl's, which has unlimited DRAM. This discrepancy stems from Orcl's insufficient grasp of the ANNS algorithm and its behavior, which results in considerable data movement overhead during data transfer between local memory and the processor complex. Additional details can be found in Figure 25, depicting the volume of data transfer via PCIe for the CXL-based systems. The figure shows that EPAX eliminates data vector transfer, thereby cutting down data transfer by 21.1 $\times$.

Furthermore, Cache improves EPAX's graph traversal time by 3.3 $\times$, thereby enhancing the query latency by an average of 32.7%. This improvement arises because Cache retains information about nodes anticipated to be accessed frequently in the local DRAM, thereby handling 59.4% of graph traversal within the local DRAM (Figure 26). The figure reveals a particularly high ratio for BigANN and Yandex-T, at 92.0%. As indicated in Table 2, their graphs have a relatively small number of neighbors (31.6 and 29.0, respectively), resulting in their graphs being compact at an average of 129.3 GB. In contrast, merely 13.8% of Meta-S's graph accesses are serviced from local memory, attributable to its extensive graph. Nevertheless, even for Meta-S, Cache enhances graph traversal performance by prefetching graph information before actual visitation as described in Section 5.2. As depicted in Figure 24, this prefetching can conceal CXL's prolonged latency, reducing Meta-S's

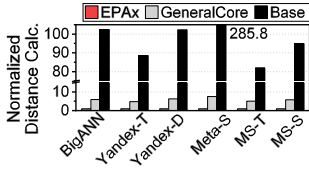


Fig. 28. Distance calculation.

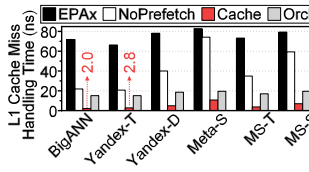


Fig. 29. Cache miss handling time.

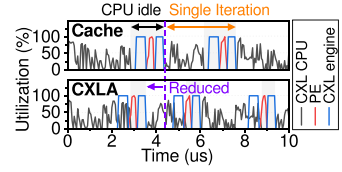


Fig. 30. Utilization (Yandex-D).

graph traversal latency by 72.8%. While the proposed prefetching would introduce overhead in speculating the next node visit, it is insignificant, accounting for only 1.3% of the query latency. These caching and prefetching techniques yield graph processing performance similar to that of Orcl. We will explain the details of prefetching shortly.

Lastly, as depicted in Figure 22, CXLA boosts the QPS by 15.5% in comparison to Cache. This is due to CXLA's enhancement of hardware resource utilization by executing deferrable subtasks and distance calculations concurrently in the CXL CPU and PE, respectively. As illustrated in Figure 24, such scheduling benefits Yandex-T, Yandex-D, and Meta-S more so than others. This is attributable to their use of a candidate array that is, on average, 16.3 $\times$ larger than others, which allows for the overlap of updates with distance calculation time. Overall, CXLA attains a significantly higher QPS than Orcl, surpassing it by an average factor of 3.8 $\times$.

6.3 Collaborative Query Service Analysis

Interface. Figure 27 illustrates the latency associated with the host-EP interface during query processing. Since this latency contributes to the total query processing time, reduced latency is preferable for optimal performance. We evaluate scenarios where DMA and MMIO are used to forward the distance calculation request to all linked EPs. In addition, we assess instances where the interface reserves its necessary memory space at either the user-level or kernel-level (indicated by suffixes -U and -K, respectively). For clarity, we averaged the figures across all datasets and presented them relative to the DMA-U result.

One can observe from the figure that the DMA-K interface manifests a 60.8% decrease in overhead when compared with the MMIO-K interface. The underlying cause of this distinction is the degree of CPU involvement. With an MMIO-based interface, the CPU must personally dispatch the commands and the neighboring list to each EP, thereby experiencing the CXL latency for every 64B of transferred data. Conversely, a DMA-oriented interface mandates the CPU to only transmit diminutive data (e.g., doorbell) to the EPs. This not only reduces the CPU's workload but also enables multiple EPs to concurrently fetch the command and neighbor list, effectively diminishing the latency. Furthermore, DMA-U trims down the DMA-K latency by an additional 19.2%, attributed to the avoidance of kernel mode switch overhead during execution.

Figure 28 contrasts the time taken for distance calculations across various methods. The novel scheme, termed GeneralCore, undertakes distance computation in the EP using a general-purpose core instead of the suggested ANNS PE. The core in the EP mirrors the configuration found in the host to distinctly highlight the implications of computations within the EP. For each dataset under consideration, values were normalized against the metric of EPAX.

From the data, it is evident that GeneralCore diminishes the distance calculation duration by an average factor of 21.4 $\times$ when set against the Base scheme. This marked enhancement primarily arises due to the minimized volume of data conveyed via the CXL network. Among all datasets, Meta-S registers the most pronounced acceleration at 37.9 $\times$. This can be attributed to Meta-S boasting the highest reduction rate of 256, as depicted in Figure 11(b). Conversely, MS-T and MS-S show minimal acceleration, a result of their lower reduction rate of 100.

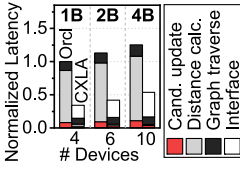


Fig. 31. Device scaling (Yandex-D).

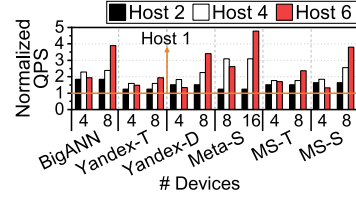


Fig. 32. Host scaling.

EPAX further amplifies the speed of distance calculations, clocking in at an average of $5.9\times$ faster than GeneralCore. This efficiency stems from the capability of our ANNS PE to concurrently manage element-wise operations and reductions for multiple vector elements. It is important to note that our EP incorporates five ANNS PEs, designed to concurrently process distance calculation requests from several host threads.

Prefetching. Figure 29 compares the L1 cache miss handling latency while accessing the graph for the CXL-based systems we tested. We measured the latency by dividing the total L1 cache miss handling time of CXL CPU by the number of L1 cache access. The new system, NoPrefetch, disables the prefetching from Cache. As shown in Figure 29, EPAX's latency is as long as 75.4 ns since it accesses slow CXL memory whenever there is a cache miss. NoPrefetch alleviates such problem thanks to local caching, shortening the latency by 45.8%. However, when the dataset uses a large graph (e.g., Meta-S, MS-S), only 24.5% of the graph can be cached in local memory. This makes NoPrefetch's latency $2.3\times$ higher than that of Orcl. In contrast, Cache significantly shortens the latency by $8.5\times$ which is even shorter than that of Orcl. This is because Cache can foresee the next visiting nodes and loads the graph information in the cache in advance. Note that, Orcl accesses local DRAM on demand on cache miss.

Utilization. Figure 30 shows the utilization of CXL CPU, PE, CXL engine on a representative dataset (Yandex-D). To clearly provide the behavior of our fine-granule scheduling, we composed a CXL-ANNS with single-core CXL CPU and single PE per device and show their behavior in a timeline. The upper part of the figure shows the behavior of Cache that does not employ the proposed scheduling. We plot CXL CPU's utilization as 0 when it polls the distance calculation results of PE, since it does not perform any useful job during that time. As shown in the figure, CXL CPU idles for 42.0% of the total time waiting for the distance calculation result. In contrast, CXLA reduces the idle time by $1.3\times$, relaxing the dependency between ANNS subtasks. In the figure, we can see that the CXL CPU's candidate update time overlaps with the time CXL engine and PE handling the command. As a result, CXLA improves the utilization of hardware resources in CXL network by 20.9%, compared to Cache.

6.4 Scalability Test

Bigger Dataset. To evaluate the scalability of CXL-ANNS, we increase the number of data vectors in Yandex-D by 4 B, and connect more EP to CXL CPU to accommodate their data. Since there is no publicly available dataset that is as large as 4 B, we synthetically generated additional 3 B vectors by adding noise to original 1B vectors. As shown in Figure 31, we can see that the latency of Orcl increases as we increase the scale of the dataset. This is because a larger dataset makes BFS visit more nodes to maintain the same level of recall. On the other hand, we can see the interface overhead of CXLA increases as we employ more devices to accommodate bigger dataset. This is because the CXL CPU should notify more devices for the command arrival by ringing the doorbell. Despite such overhead, CXLA exhibits $2.7\times$ lower latency than Orcl thanks to its efficient collaborative approach.

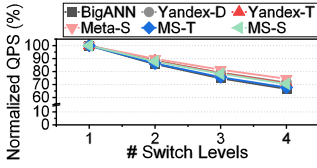


Fig. 33. Multi-level switch.

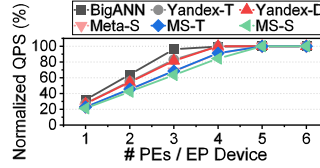


Fig. 34. Number of PEs.

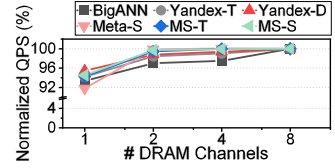


Fig. 35. Number of channels.

Multi-host. In a disaggregated system, a natural way to increase the system's performance is to employ more host CPUs. Thus, we evaluate the CXL-ANNS that supports multiple hosts in the CXL network. Specifically, we split EP's resources such as HDM and PEs and then allocate each of them to one the CXL hosts in the network. For ANNS, we partition the embedding table and make each host responsible for finding kNN from different partitions. Once all the CXL hosts find the kNN, the system gathers them all and rerank the neighbors to finally select kNN among them.

Figure 32 shows the QPS of multi-host ANNS. The QPS is normalized to that when we use a single CXL host with the same number of EPs that we used before. Note that we also show the QPS when we employ more number of EPs than we used before. When the number of EPs stays the same, the QPS increases until we connect 4 CXL hosts in the system. However the QPS drops when the number of CXL hosts is 6. This is because the distance calculation by limited number of PEs became the bottleneck; the commands from the host pends since there is no available PE. Such problem can be addressed by having more EPs in the system, thereby distributing the computation load. As we can see in the figure, when we double the number of EPs in the network, we can improve the QPS when we have 6 CXL hosts in the system.

Multi-level switch. To support larger datasets, systems based on CXL 3.0 can extend their memory capacity by integrating multiple switches between the host and the EPs, leveraging the multi-level switch feature of CXL 3.0. However, the incorporation of multiple switches might negatively influence overall system performance. Each additional switch contributes an added latency of 75~130 ns to the communication pathway between the host and the EPs [20, 40]. To understand this impact, we assessed the performance of CXL-ANNS by varying the switch levels positioned between the host and EPs, with results presented in Figure 33. For every dataset, the values were standardized against scenarios using a singular switch.

From the data, we observe that when the switch levels escalate from 1 to 4, the QPS of CXL-ANNS diminishes by an average factor of 1.4×. The degradation's magnitude varies based on the dataset. For instance, in Meta-S, the performance drop with 4 switch levels is 1.3×, the least across all datasets. This resilience can be attributed to Meta-S having a high embedding dimension and average neighbor count (256 and 190, respectively), which reduces the relative impact of communication on overall latency. On the other hand, BigANN displays the most pronounced performance reduction at 1.5×, stemming from its lower embedding dimension and average neighbor figure (128 and 31.6, respectively). Nonetheless, even with as many as 4 switch levels, CXL-ANNS outperforms Orcl, a testament to its efficiency in distance calculation within the EP and local graph caching strategies.

6.5 Sensitivity Analysis

This section outlines our process for determining the essential architectural configurations for EPs, specifically focusing on: (i) The quantity of ANNS PEs, and (ii) The number of DRAM channels.

Number of PEs. The quantity of PEs plays a pivotal role in determining both the overall system performance and the resource efficiency. Operating with an insufficient number of PEs can cause

lags when processing distance calculation requests. On the other hand, an overly abundant PE count might lead to underutilized or idle PEs. For CXL-ANNS, we ascertained the optimal PE count by experimenting with systems that had varying PE quantities. The outcomes are depicted in Figure 34. For clarity, we normalized the values using a benchmark system that houses six PEs per EP device. Even though the QPS reaches saturation across all datasets when five PEs are deployed per EP, the performance nuances differ among datasets. For instance, BigANN attains peak QPS with a mere three PEs per EP device, attributable to its modest average neighbor count of 31.6, which necessitates minimal distance computation. Conversely, MS-S requires five PEs to achieve saturation due to its nodes averaging 87.4 neighbors, resulting in a higher computational demand for distance metrics. Intriguingly, while Meta-S boasts the most neighbors (averaging 190), it also saturates with a similar PE count. This is attributed to Meta-S dedicating more time on the host side, handling an extensive candidate array of approximately 5,600 nodes. Such management renders distance calculation requests to the EP less frequent.

DRAM channel count within an EP. The quantity of DRAM channels can significantly influence performance, given that it dictates how ANNS retrieves embedding vectors and graph structures for searches. Figure 35 presents the throughput of CXL-ANNS as the number of DRAM channels fluctuates between one and eight. The displayed results are calibrated against the performance when eight channels are employed. Remarkably, EPs with a mere two DRAM channels can match the performance of those using eight channels, showcasing a minimal average discrepancy of just 1.1%. Consequently, we set the DRAM channel count in an EP to two for our architectural design.

7 DISCUSSION

Managing cache coherence. Calculating distances within the EP can effectively diminish communication overhead by limiting the data volume transmitted through the CXL network. However, this approach might produce inaccurate results if the host modifies the embedding vector during runtime. Such discrepancies arise because the EP could retrieve an outdated embedding vector, especially if the refreshed vector is cached within the CPU.

Fortunately, standard ANNS services infrequently adjust the embedding table, maintaining it in a read-only state for the majority of its operational time [15]. Taking a cue from [34], CXL-ANNS restricts modifications to the embedding table during its runtime. Specifically, if the CXL engine within the EP receives a write request directed towards the embedding table, it halts the request. To facilitate this, our EP obtains the embedding table's address from the kernel driver via an interface register. This address is then relayed to the CXL engine, which subsequently denies any write requests targeting that location.

Though present-day ANNS rarely alters the embedding table, we recognize that facilitating real-time updates to the embedding table could be an intriguing avenue for future research. We earmark this consideration for subsequent exploration.

GPU-based distance calculation. Recent research has begun to leverage the massive parallel processing capabilities of GPUs to enhance the efficiency of graph-based ANNS services [68, 73]. While GPUs generally exhibit high performance, our argument is that it's not feasible for CPU+GPU memory to handle the entirety of ANNS data and tasks, as detailed in Section 1. Even under the assumption that ANNS is functioning within an optimal in-memory computing environment, there are two elements to consider when delegating distance computation to GPUs. The first point is that GPUs require interaction with the host's software and/or hardware layers, which incurs a data transfer overhead for computation. Secondly, ANNS distance computations can be carried out using a few uncomplicated, lightweight vector processing units, making GPUs a less cost-efficient choice for these distance calculation tasks.

In contrast, CXL-ANNS avoids the burden of data movement overhead, as it processes data in close proximity to its actual location and returns only a compact result set. This approach to data processing is well established and has been validated through numerous application studies [1, 25, 33, 36–38, 49, 53]. Moreover, CXL-ANNS effectively utilizes the cache hierarchy and can even decrease the frequency of accesses to the underlying CXL memory pool. It accomplishes this through its CXL-aware and ANNS-aware prefetching scheme, which notably enhances performance.

Domain-specific acceleration. There exists a trend of utilizing domain-specific acceleration to optimize ANNS services [39, 43, 54, 69]. FAERY [69] introduces an acceleration framework tailored for brute-force kNN searches. To facilitate high bandwidth during embedding table retrievals, the embedding table is stored in the HBM directly connected to the accelerator. This accelerator features multiple distance calculation units and a K-selection unit, which systematically chooses the k closest vectors from the entire set of embedding vectors.

Conversely, QuickNN [54] focuses on enhancing the KD tree-based ANNS. By scrutinizing the memory access behaviors of the KD tree structure, QuickNN strategically places the frequently accessed non-leaf segments of the tree in an on-chip cache. Meanwhile, the leaf portions of the tree are stored in off-chip DRAM. Additionally, QuickNN incorporates several tree traversal units to concurrently process incoming queries.

Although both FAERY and QuickNN offer potent acceleration capabilities for search services, they have a limitation. They necessitate both the embedding table and the related index structure to be accommodated within the accelerator’s memory. As such, they may not be suitable for expansive, large-scale ANNS services.

Large-scale ANNS. To this end, several research initiatives have aimed at facilitating acceleration for large-scale ANNS. ANNA [39] expedites the compression method for ANNS by fully automating the product quantization process [32]. Recognizing that multiple neighbors linked to the same centroid possess shared distance calculation outcomes, ANNA caches these results to minimize computational requirements. Additionally, it is equipped to handle both L2 and angular distance metrics. Yet, due to the inherent compromises of compression, ANNA’s search accuracy is compromised, as detailed in Section 2.2.

Vstore [43], on the other hand, introduces an in-storage graph-based ANNS acceleration strategy. This framework automates the best-first search algorithm within a NAND flash-centric SSD, using its internal DRAM as a caching mechanism. In a bid to curtail the number of sluggish NAND flash accesses, Vstore processes queries in succession, especially those likely to explore a shared neighbor set. This approach augments the likelihood of the pertinent vectors being managed within the internal DRAM. However, Vstore’s performance is still hampered by the extended latency characteristic of NAND flash accesses.

Conversely, CXL-ANNS capitalizes on the CXL memory pool to furnish ample memory space, negating the need for compression and thereby ensuring a precise search service. Moreover, our integrated software-hardware approach markedly enhances the efficacy of kNN searches.

CXL. As CXL garners increasing interest within the industry, several initiatives focusing on CXL-driven memory pooling have emerged [20, 40, 50]. DirectCXL [20] pioneers the practical design of CXL 2.0-based memory pooling, grounded in an FPGA-centric platform. It specifically introduces designs for the operating system and hardware elements (like the CXL root port, switch, and endpoint) to facilitate memory pooling. For validation and performance analysis, DirectCXL employs CXL memory as working memory while assessing real-world applications, such as recommendation systems, key-value stores, and graph analysis. Although this showcases CXL’s instruction-level compatibility, there remains a potential for performance enhancements, especially by leveraging faster local DRAM resources.

Pond [40] addresses this performance gap in a public cloud context, utilizing a machine learning-driven memory allocator that gauges the latency sensitivity of diverse VMs. If a VM is identified as latency-sensitive, Pond prioritizes local DRAM over CXL memory for that VM's working memory allocation. This strategy mitigates the extended latency introduced by accessing the CXL memory pool. However, while effective for latency-sensitive tasks with minimal memory demands, it is not universally applicable, especially for applications needing memory capacities exceeding a single machine's offering. While a straightforward approach might involve maximizing local memory usage before resorting to CXL memory, it could lead to suboptimal performance due to unaddressed data locality nuances. TPP [50] offers a solution by proposing a data placement strategy that recognizes data locality at page-level granularity. In essence, TPP moves less frequently accessed pages to CXL memory and more frequently accessed ones to local memory. Implemented at the kernel level, TPP offers users a seamless interface. However, it does not fully optimize performance for applications like ANNS, as it fails to discern fine-grained locality attributes inherent in certain data structures.

Differently, CXL-ANNS optimally harnesses CXL-based systems by implementing graph local caching, factoring in each node's locality within the graph. Consequently, CXL-ANNS not only benefits from the expansive memory capacity of the CXL memory pool but also minimizes latency challenges tied to flit conversion. Furthermore, by prefetching graph data and conducting distance computations within EPs, CXL-ANNS surpasses even oracle system performance benchmarks.

Graph-based ANNS algorithms. Recently, numerous graph-based ANNS algorithms have been developed to enhance the efficiency of kNN searches. Although they all typically create a graph using data vectors as nodes, their policies for node connection differ. Here, we spotlight three leading algorithms adopted by different corporate production services [7, 15, 70].

The **Navigating Spreading-out Graph (NSG [15])** approach emphasizes forming essential links to pave a route from an entry node to all other nodes. Conversely, Vamana [30] seeks to minimize the hops required to move from the entry node to others. It achieves this by discarding connections between proximal nodes and linking those more distant from each other. The **Hierarchical Navigable Small World (HNSW [48])** methodology further integrates a streamlined graph, akin to HM-ANN, offering shortcuts to nodes distant from the entry points.

Even though these algorithms differ in graph construction, they all adopt a similar traversal technique: a best-first search that originates from a set entry node. CXL-ANNS sidesteps algorithm-specific connection policies, choosing instead to capitalize on the widespread best-first search approach. Hence, we anticipate that CXL-ANNS can broadly enhance most graph-based ANNS algorithms, delivering swifter results than an oracle system. Furthermore, CXL-ANNS can also optimize graph construction. The construction algorithm typically reviews the precise kNN for all nodes, a process that can be expedited using ANNS PEs. Exploring acceleration in graph construction remains an area for future investigation.

8 CONCLUSION

We propose CXL-ANNS, a software-hardware collaborative approach for scalable ANNS. CXL-ANNS places all the dataset into its CXL memory pool to handle billion-point graphs while making the performance of the kNN search comparable with that of the (local-DRAM only) oracle system. To this end, CXL-ANNS considers inter-node relationship and performs ANNS-aware prefetches. It also calculate distances in its EP while scheduling the ANNS subtasks to utilize all the resources in the CXL network. Our empirical results show that CXL-ANNS exhibits 111.1× better performance compared to the state-of-the-art billion-scale ANNS methods and 3.8× better performance than oracle system, respectively.

REFERENCES

- [1] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoun Choi. 2015. A scalable processing-in-memory accelerator for parallel graph processing. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA'15)*.
- [2] Michael J. Anderson, Benny Chen, Stephen Chen, Summer Deng, Jordan Fix, Michael Gschwind, Aravind Kalaiah, Changkyu Kim, Jaewon Lee, Jason Liang, Haixin Liu, Yinghai Lu, Jack Montgomery, Arun Moorthy, Nadathur Satish, Sam Naghshineh, Avinash Nayak, Jongsoo Park, Chris Petersen, Martin Schatz, Narayanan Sundaram, Bangsheng Tang, Peter Tang, Amy Yang, Jiecao Yu, Hector Yuen, Ying Zhang, Aravind Anbudurai, Vandana Balan, Harsha Bojja, Joe Boyd, Matthew Breitbach, Claudio Caldato, Anna Calvo, Garret Catron, Sneha Chandwani, Panos Christeas, Brad Cottle, Brian Coutinho, Arun Dalli, Abhishek Dhanotia, Oniel Duncan, Roman Dzhabarov, Simon Elmir, Chunli Fu, Wenying Fu, Michael Fulthorp, Adi Gangidi, Nick Gibson, Sean Gordon, Beatriz Padilla Hernandez, Daniel Ho, Yu-Cheng Huang, Olof Johansson, Shishir Juluri, Shobhit Kanaujia, Manali Kesarkar, Jonathan Killinger, Ben Kim, Rohan Kulkarni, Meghan Lele, Huayu Li, Huamin Li, Yueming Li, Cynthia Liu, Jerry Liu, Bert Maher, Chandra Mallipedi, Seema Mangla, Kiran Kumar Matam, Jubin Mehta, Shobhit Mehta, Christopher Mitchell, Bharath Muthiah, Nitin Nagarkatte, Ashwin Narasimha, Bernard Nguyen, Thiara Ortiz, Soumya Padmanabha, Deng Pan, Ashwin Poojary, Ye (Charlotte) Qi, Olivier Raginel, Dwarak Rajagopal, Tristan Rice, Craig Ross, Nadav Rotem, Scott Russ, Kushal Shah, Baohua Shan, Hao Shen, Pavan Shetty, Krish Skandakumaran, Kutta Srinivasan, Roshan Sumbaly, Michael Tauberg, Mor Tzur, Sidharth Verma, Hao Wang, Man Wang, Ben Wei, Alex Xia, Chenyu Xu, Martin Yang, Kai Zhang, Ruoxi Zhang, Ming Zhao, Whitney Zhao, Rui Zhu, Ajit Mathew, Lin Qiao, Misha Smelyanskiy, Bill Jia, and Vijay Rao. 2021. First-generation inference accelerator deployment at Facebook. arXiv:2107.04140. Retrieved from <https://arxiv.org/abs/2107.04140>
- [3] Sunil Arya and David M. Mount. 1993. Approximate nearest neighbor queries in fixed dimensions. In *Proceedings of the 4th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA'93)*.
- [4] Sunil Arya, David M. Mount, Nathan S. Netanyahu, Ruth Silverman, and Angela Y. Wu. 1998. An optimal algorithm for approximate nearest neighbor searching fixed dimensions. *Journal of the ACM* 45, 6 (1998), 891–923.
- [5] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2017. ANN-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. In *Proceedings of the International Conference on Similarity Search and Applications*. Springer.
- [6] Artem Babenko and Victor Lempitsky. 2014. The inverted multi-index. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 37, 6 (2014), 1247–1260.
- [7] Paul Baltescu, Haoyu Chen, Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. ItemSage: Learning product embeddings for shopping recommendations at pinterest. arXiv:2205.11728. Retrieved from <https://arxiv.org/abs/2205.11728>
- [8] Christopher Celio, Pi-Feng Chiu, Borivoje Nikolic, David A. Patterson, and Krste Asanovic. 2017. BOOMv2: An open-source out-of-order RISC-V core. In *Proceedings of the First Workshop on Computer Architecture Research with RISC-V (CARRV'17)*.
- [9] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient billion-scale approximate nearest neighbor search. In *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS'21)*.
- [10] Rihan Chen, Bin Liu, Han Zhu, Yaoyuan Wang, Qi Li, Buting Ma, Qingbo Hua, Jun Jiang, Yunlong Xu, Hongbo Deng, and Bo Zheng. 2022. Approximate nearest neighbor search under neural similarity metric for large-scale recommendation. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM'22)*.
- [11] CXL Consortium. 2022. Compute Express Link 3.0 White Paper. Retrieved 30 January 2024 from https://www.computeexpresslink.org/_files/ugd/0c1418_a8713008916044ae9604405d10a7773b.pdf
- [12] Guohao Dai, Tianhao Huang, Yuze Chi, Ningyi Xu, Yu Wang, and Huazhong Yang. 2017. ForeGraph: Exploring large-scale graph processing on multi-FPGA architecture. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA'17)*.
- [13] Jeffrey Dean and Luiz André Barroso. 2013. The tail at scale. *Communications of the ACM* 56, 2 (2013), 74–80.
- [14] Karima Echihiabi, Kostas Zoumpatianos, Themis Palpanas, and Houda Benbrahim. 2019. Return of the lernaean hydra: Experimental evaluation of data series approximate similarity search. *Proceedings of the VLDB Endowment* 13, 3 (2019), 403–420.
- [15] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proceedings of the VLDB Endowment* 12, 5 (2019), 461–474.
- [16] Vincent Garcia, Eric Debreuve, and Michel Barlaud. 2008. Fast k-nearest neighbor search using GPU. In *Proceedings of the 2008 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*. IEEE.
- [17] Tiezheng Ge, Kaoming He, Qifa Ke, and Jian Sun. 2013. Optimized product quantization for approximate nearest neighbor search. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR'13)*.

- [18] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity search in high dimensions via hashing. In *Proceedings of the VLDB*.
- [19] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023 (WWW'23)*.
- [20] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. 2022. Direct access, {High-Performance} memory disaggregation with {DirectCXL}. In *Proceedings of the 2022 USENIX Annual Technical Conference (USENIX ATC'22)*. 287–294.
- [21] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the International Conference on Machine Learning (ICML'20)*. Retrieved from <https://arxiv.org/abs/1908.10396>
- [22] Udit Gupta, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagen, David Brooks, Bradford Cottel, Kim Hazelwood, Mark Hempstead, Bill Jia, Hsien-Hsin S. Lee, Andrey Malevich, Dheevatsa Mudigere, Mikhail Smelyanskiy, Liang Xiong, and Xuan Zhang. 2020. The architectural implications of facebook's dnn-based personalized recommendation. In *Proceedings of the 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA'20)*. IEEE, 488–501.
- [23] Kiana Hajebi, Yasin Abbasi-Yadkori, Hossein Shahbazi, and Hong Zhang. 2011. Fast approximate nearest-neighbor search with k-nearest neighbor graph. In *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI'11)*.
- [24] John A. Hartigan and Manchek A. Wong. 1979. Algorithm AS 136: A k-means clustering algorithm. *Journal of the Royal Statistical Society. Series c (Applied Statistics)* 28, 1 (1979), 100–108.
- [25] Jun Heo, Seung Yul Lee, Sunhong Min, Yeonhong Park, Sung Jun Jung, Tae Jun Ham, and Jae W. Lee. 2021. Boss: Bandwidth-optimized search accelerator for storage-class memory. In *Proceedings of the 2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA'21)*. IEEE.
- [26] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. 2020. Embedding-based retrieval in Facebook search. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'20)*.
- [27] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. 2015. Query-aware locality-sensitive hashing for approximate nearest neighbor search. *Proceedings of the VLDB Endowment* 9, 1 (2015), 1–12.
- [28] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*.
- [29] Intel. 2021. Optane SSD 9 Series. Retrieved 30 January 2024 from <https://www.intel.com/content/www/us/en/products/details/memory-storage/consumer-ssds/optane-ssd-9-series.html>
- [30] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Proceedings of the Advances in Neural Information Processing Systems (NeurIPS'19)*. H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.). Retrieved from <https://proceedings.neurips.cc/paper/2019/file/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Paper.pdf>
- [31] Herve Jegou, Matthijs Douze, Jeff Johnson, Lucas Hosseini, Chengqi Deng, and Alexandr Guzhva. 2018. Faiss. Retrieved 30 January 2024 from <https://github.com/facebookresearch/faiss>
- [32] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2010), 117–128.
- [33] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S Lee, et al. 2020. Recnmp: Accelerating personalized recommendation with near-memory processing. In *Proceedings of the 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA'20)*. IEEE.
- [34] Liu Ke, Xuan Zhang, Jinin So, Jong-Geon Lee, Shin-Haeng Kang, Sukhan Lee, Songyi Han, YeonGon Cho, Jin Hyun Kim, Yongsuk Kwon, KyungSoo Kim, Jin Jung, Ilkwon Yun, Sung Joo Park, Hyunsun Park, Joonho Song, Jeonghyeon Cho, Kyomin Sohn, Nam Sung Kim, and Hsien-Hsin S. Lee. 2021. Near-memory processing in action: Accelerating personalized recommendation with axdim. *IEEE Micro* 42, 1 (2021), 116–127.
- [35] Kenneth C. Knowlton. 1965. A fast storage allocator. *Communications of the ACM* 8, 10 (1965), 623–624. DOI:<https://doi.org/10.1145/365628.365655>
- [36] Miryeong Kwon, Donghyun Gouk, Sangwon Lee, and Myoungsoo Jung. 2022. Hardware/software co-programmable framework for computational SSDs to accelerate deep learning service on large-scale graphs. In *Proceedings of the 20th USENIX Conference on File and Storage Technologies (FAST'22)*.

- [37] Miryeong Kwon, Junhyeok Jang, Hanjin Choi, Sangwon Lee, and Myoungsoo Jung. 2023. Failure tolerant training with persistent memory disaggregation over CXL. *IEEE Micro* 43, 2 (2023), 66–75.
- [38] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. 2019. Tensordimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'19)*.
- [39] Yejin Lee, Hyunji Choi, Sunhong Min, Hyunseung Lee, Sangwon Beak, Dawoon Jeong, Jae W. Lee, and Tae Jun Ham. 2022. ANNA: Specialized architecture for approximate nearest neighbor search. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture, HPCA 2022*. IEEE, 169–183. DOI: <https://doi.org/10.1109/HPCA53966.2022.00021>
- [40] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. 2023. Pond: CXL-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.
- [41] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2019. Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2019), 1475–1488.
- [42] Chieh-Jan Mike Liang, Hui Xue, Mao Yang, Lidong Zhou, Lifei Zhu, Zhao Lucis Li, Zibo Wang, Qi Chen, Quanlu Zhang, Chuanjie Liu, and Wenjun Dai. 2020. AutoSys: The design and operation of learning-augmented systems. In *Proceedings of the 2020 USENIX Conference on Usenix Annual Technical Conference*. 323–336.
- [43] Shengwen Liang, Ying Wang, Ziming Yuan, Cheng Liu, Huawei Li, and Xiaowei Li. 2022. VStore: In-storage graph based vector search accelerator. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 997–1002.
- [44] Linaro. 2023. The Devicetree Specification. Retrieved 30 January 2024 from <https://www.devicetree.org/>
- [45] Jiawen Liu, Zhen Xie, Dimitrios Nikolopoulos, and Dong Li. 2020. RIANN: Real-time incremental learning with approximate nearest neighbor on mobile devices. In *Proceedings of the 2020 USENIX Conference on Operational Machine Learning (OpML'20)*.
- [46] Ting Liu, Andrew Moore, Ke Yang, and Alexander Gray. 2004. An investigation of practical approximate nearest neighbor algorithms. In *Proceedings of the Advances in Neural Information Processing Systems (NIPS'04)*. L. Saul, Y. Weiss, and L. Bottou (Eds.). Retrieved from <https://proceedings.neurips.cc/paper/2004/file/1102a326d5f7c9e04fc3c89d0ede88c9-Paper.pdf>
- [47] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, Adria Arnejach, Nils Asmussen, Srikant Bharadwaj, Gabe Black, Gedare Bloom, Bobby R. Bruce, Daniel Rodrigues Carvalho, Jerónimo Castrillón, Lizhong Chen, Nicolas Derumigny, Stephan Diestelhorst, Wendy Elsasser, Marjan Fariborz, Amin Farmahini Farahani, Pouya Fotouhi, Ryan Gambord, Jayneel Gandhi, Dibakar Gope, Thomas Grass, Bagus Hanindhito, Andreas Hansson, Swapnil Haria, Austin Harris, Timothy Hayes, Adrian Herrera, Matthew Horsnell, Syed Ali Raza Jafri, Radhika Jagtap, Hanhwi Jang, Reiley Jeyapaul, Timothy M. Jones, Matthias Jung, Subash Kannoth, Hamidreza Khaleghzadeh, Yuetsu Kodama, Tushar Krishna, Tommaso Marinelli, Christian Menard, Andrea Mondelli, Tiago Mück, Omar Naji, Krishnendra Nathella, Hoa Nguyen, Nikos Nikoleris, Lena E. Olson, Marc S. Orr, Binh Pham, Pablo Prieto, Trivikram Reddy, Alec Roelke, Mahyar Samani, Andreas Sandberg, Javier Setoain, Boris Shingarov, Matthew D. Sinclair, Tuan Ta, Rahul Thakur, Giacomo Travaglini, Michael Upton, Nilay Vaish, Ilias Vougioukas, Zhengrong Wang, Norbert Wehn, Christian Weis, David A. Wood, Hongil Yoon, and Éder F. Zulian. 2020. The gem5 simulator: Version 20.0+. arXiv:2007.03152. Retrieved from <https://arxiv.org/abs/2007.03152>
- [48] Yu A. Malkov and Dmitry A. Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2018), 824–836.
- [49] Nika Mansouri Ghiasi, Jisung Park, Harun Mustafa, Jeremie Kim, Ataberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoun Alserr, Rachata, Nandita Vijaykumar, Mohammed Alser, and Onur Mutlu. 2022. GenStore: A high-performance in-storage processing system for genome sequence analysis. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'22)*.
- [50] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. 2023. TPP: Transparent page placement for CXL-enabled tiered-memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 742–755.
- [51] Marius Muja and David G. Lowe. 2014. Scalable nearest neighbor algorithms for high dimensional data. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 11 (2014), 2227–2240.
- [52] Pandu Nayak. 2019. Understanding Searches Better than Ever Before. Retrieved from <https://blog.google/products/search/search-language-understanding-bert/>

- [53] Joel Nider, Craig Mustard, Andrada Zoltan, John Ramsden, Larry Liu, Jacob Grossbard, Mohammad Dashti, Romaric Jodin, Alexandre Ghit, Jordi Chauzi, and Alexandra Fedorova. 2021. A case study of processing-in-memory in off-the-shelf systems. In *Proceedings of the USENIX Annual Technical Conference (ATC'21)*. 117–130.
- [54] Reid Pinkham, Shuqing Zeng, and Zhengya Zhang. 2020. Quicknn: Memory and performance optimization of kd tree based nearest neighbor search for 3d point clouds. In *Proceedings of the 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA'20)*. IEEE, 180–192.
- [55] Shafiu Rahman, Nael Abu-Ghazaleh, and Rajiv Gupta. 2020. Graphpulse: An event-driven hardware accelerator for asynchronous graph processing. In *Proceedings of the 2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'20)*. IEEE.
- [56] Jie Ren, Minjia Zhang, and Dong Li. 2020. Hm-ann: Efficient billion-point nearest neighbor search on heterogeneous memory. *Advances in Neural Information Processing Systems* 33 (2020), 10672–10684.
- [57] Harsha Simhadri. 2021. Research Talk: Approximate Nearest Neighbor Search Systems at Scale. Retrieved 30 January 2024 from <https://www.youtube.com/watch?v=BnYNdSIKibQ&list=PLD7HFcN7LXReJTWfKYqWMcCc1nZKIXBo9&index=9>
- [58] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. 2022. Results of the NeurIPS'21 challenge on billion-scale approximate nearest neighbor search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*. PMLR.
- [59] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A fast and accurate graph-based ANN index for streaming similarity search. arXiv:2105.09613. Retrieved from <https://arxiv.org/abs/2105.09613>
- [60] Yifang Sun, Wei Wang, Jianbin Qin, Ying Zhang, and Xuemin Lin. 2014. SRS: Solving c-approximate nearest neighbor queries in high dimensional euclidean space with a tiny index. *Proceedings of the VLDB Endowment* 8, 1 (2014), 1–12.
- [61] Mikkel Thorup. 1999. Undirected single-source shortest paths with positive integer weights in linear time. *Journal of the ACM* 46, 3 (1999), 362–394. DOI: <https://doi.org/10.1145/316542.316548>
- [62] Inc UEFI Forum. 2021. Advanced Configuration and Power Interface (ACPI) Specification Version 6.4. Retrieved 30 January 2024 from <https://uefi.org/specs/ACPI/6.4/>
- [63] Charlie Waldburger. 2019. As Search Needs Evolve, Microsoft Makes AI Tools for Better Search Available to Researchers and Developers. Retrieved 30 January 2024 from <https://news.microsoft.com/source/features/ai/bing-vector-search/>
- [64] Jingdong Wang, Naiyan Wang, You Jia, Jian Li, Gang Zeng, Hongbin Zha, and Xian-Sheng Hua. 2013. Trinary-projection trees for approximate nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 2 (2013), 388–403.
- [65] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxian Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD'21)*. 2614–2627.
- [66] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. arXiv:2101.12631. Retrieved from <https://arxiv.org/abs/2101.12631>
- [67] Roger Weber, Hans-Jörg Schek, and Stephen Blott. 1998. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. In *Proceedings of the VLDB*.
- [68] Yuanhang Yu, Dong Wen, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2022. GPU-accelerated proximity graph approximate nearest neighbor search and construction. In *Proceedings of the 2022 IEEE 38th International Conference on Data Engineering (ICDE'22)*. IEEE.
- [69] Chaoliang Zeng, Layong Luo, Qingsong Ning, Yaodong Han, Yuhang Jiang, Ding Tang, Zilong Wang, Kai Chen, and Chuanxiong Guo. 2022. {FAERY}: An {FPGA-accelerated} embedding-based retrieval system. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI'22)*. 841–856.
- [70] Jianjin Zhang, Zheng Liu, Weihao Han, Shitao Xiao, Ruicheng Zheng, Yingxia Shao, Hao Sun, Hanqing Zhu, Premkumar Srinivasan, Weiwei Deng, and Xing Xie. 2022. Uni-retriever: Towards learning the unified embedding based retriever in bing sponsored search. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD'22)*. 4493–4501.
- [71] Minjia Zhang and Yuxiong He. 2019. Grip: Multi-store capacity-optimized high-performance nearest neighbor search for vector search engine. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM'19)*.
- [72] Yanhao Zhang, Pan Pan, Yun Zheng, Kang Zhao, Yingya Zhang, Xiaofeng Ren, and Rong Jin. 2018. Visual search at alibaba. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'18)*.

- [73] Weijie Zhao, Shulong Tan, and Ping Li. 2020. Song: Approximate nearest neighbor search on gpu. In *Proceedings of the 2020 IEEE 36th International Conference on Data Engineering (ICDE'20)*. IEEE.
- [74] Andy Diwen Zhu, Xiaokui Xiao, Sibowang, and Wenqing Lin. 2013. Efficient single-source shortest path and distance queries on large graphs. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD 2013*. ACM. DOI:<https://doi.org/10.1145/2487575.2487665>

Received 11 October 2023; accepted 15 December 2023